

2026 TECH WEEK:

Physical AI

Physical AI의 유형

휴머노이드형



자율주행차형



드론형



무인 운반차 및
자율주행 로봇형
(AGV·AMR)



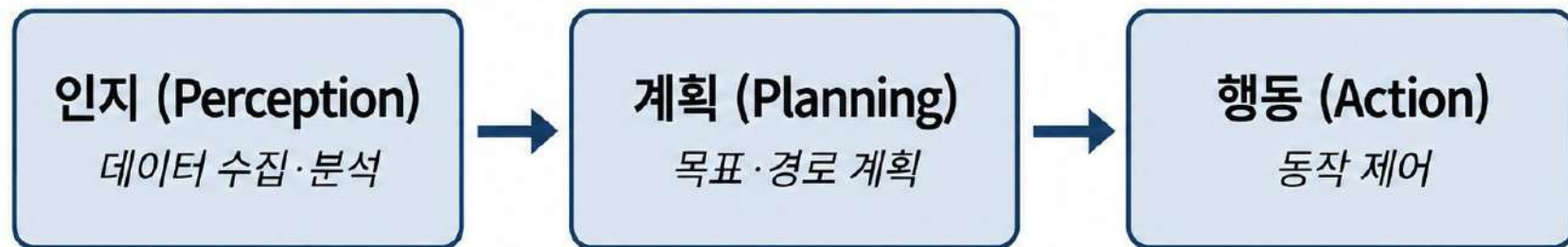
Autonomous Mobile Robot

자율 이동 로봇 (AMR)



- 실시간 환경 인식, 장애물 회피 및 최적 경로 생성
- AI, 컴퓨터 비전, LiDAR 등과 결합하여 인식·판단·제어 기능 고도화
- 공장, 물류센터, 건설현장, 병원, 공항, 캠퍼스 등 다양한 산업 및 현장에서 활용
- 인간 작업자와 로봇이 동일한 작업 공간에서 안전하게 상호작용하며 협업 수행

Autonomous Mobile Robot



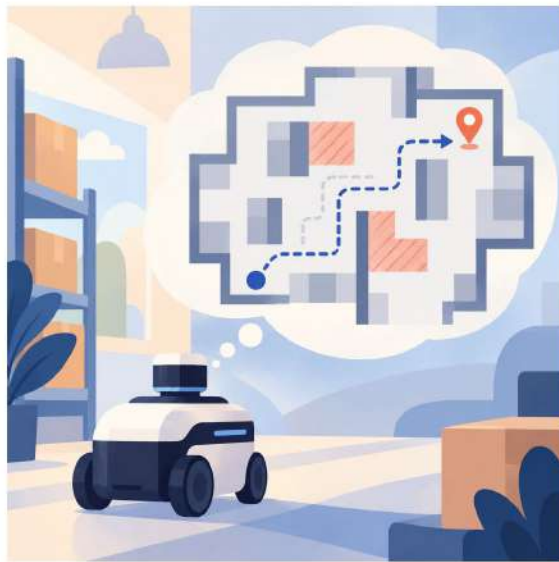
인지, Perception

- 다양한 센서를 통한 환경 데이터 수집
- 수집 데이터의 전처리 및 특징 추출
- 주변 환경 및 객체 정보 인식
- 로봇의 위치 및 주변 상태 추정



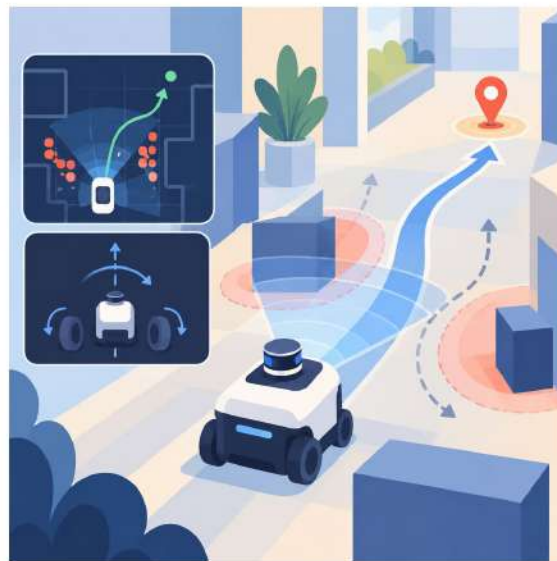
계획, Planning

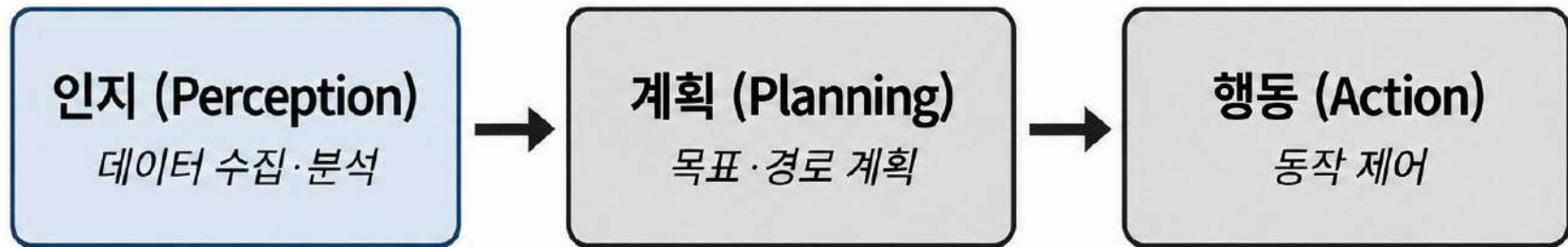
- 인지 정보를 기반으로 환경 상황 분석
- 주어진 목표에 따른 행동 계획 수립
- 목적지까지의 최적 경로 생성
- 환경 변화에 따른 계획의 실시간 갱신



행동, Action

- 계획된 경로에 따른 이동 명령 생성
- 로봇의 속도 및 방향 제어
- 주행 중 위치 및 자세 실시간 보정
- 상황 변화에 따른 로봇 동작 제어





Sensor Inputs



LiDAR

LiDAR = Light Detection and Ranging

- 레이저 기반 거리 측정
- 360도 스캔 기법



Velodyne



Sick



ibeo



Quanergy

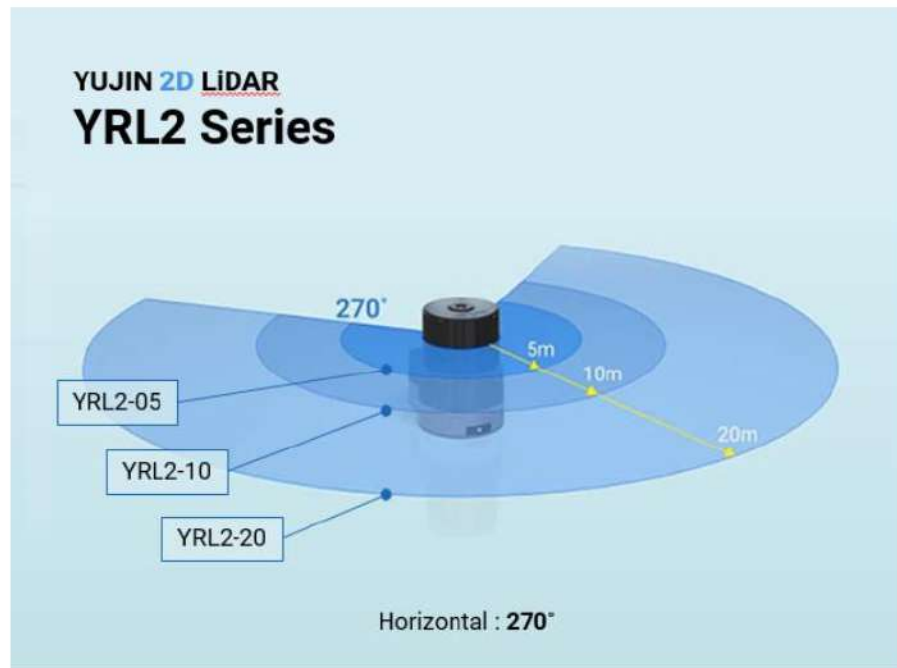


Valeo

LiDAR

LiDAR = Light Detection and Ranging

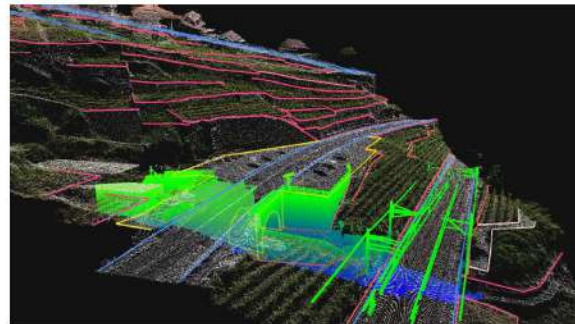
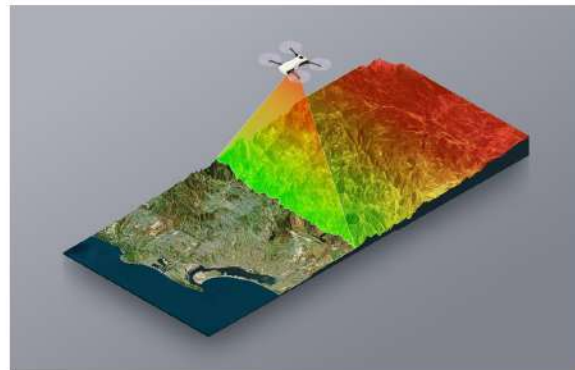
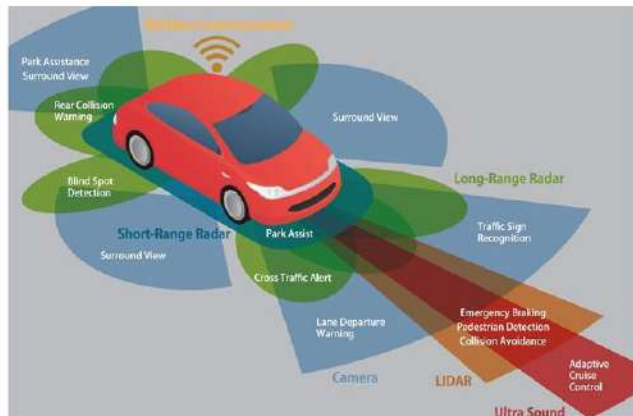
- 레이저 기반 거리 측정
- 360도 스캔 기법



LiDAR

LiDAR 활용 사례

- 자율주행 차량
- 로봇/드론
- 지형 측량
- 안전 관리



LiDAR

2D LiDAR vs. 3D LiDAR

2D LiDAR

- 특징: 수평선으로 레이저 스캔
- 데이터 형태: (X, Y) 좌표
- 용도: 2D 지도 작성, 장애물 회피, 위치 추정

3D LiDAR

- 특징: 수평 레이저를 여러 채널 (16, 32, 128 채널)
- 데이터 형태: 3차원 Point Cloud, Voxel
- 용도: 야외 자율주행, 객체 인식, 정밀 측량, 3D 모델링



LiDAR

LiDAR 데이터 구조

- 최소 각도
- 최대 각도
- 최소 거리
- 최대 거리
- 측정 개수
- 해상도
- 회전 주기
- 샘플링 주기
- 측정 값 (거리)
- 측정 값 (강도)

IMU

Inertial Measurement Unit, IMU:

- 가속도계와 자이로를 이용해 선형 가속도 및 각속도 측정
- 높은 주기로 자세·운동 상태를 추정할 수 있으나 시간에 따른 **drift** 발생
- 자력계를 이용해 지구 자기장을 측정하고 방향 정보를 추정



GNSS

Global Navigation Satellite System, GNSS:

- 위성 신호를 이용해 절대 위치 및 속도 제공
- 장시간 사용 시 **drift**가 적어 전역 좌표계 기준 위치 보정에 유리
- 건물·터널·지하 공간 등에서는 신호 차단 및 정확도 저하



Encoder

Wheel Encoder:

- 바퀴의 회전량을 측정하여 주행 거리 및 차량 속도 추정
- 저속 주행에서도 비교적 안정적인 차량 이동 정보 제공
- 타이어 슬립이나 미끄러짐 발생 시 누적 오차 증가



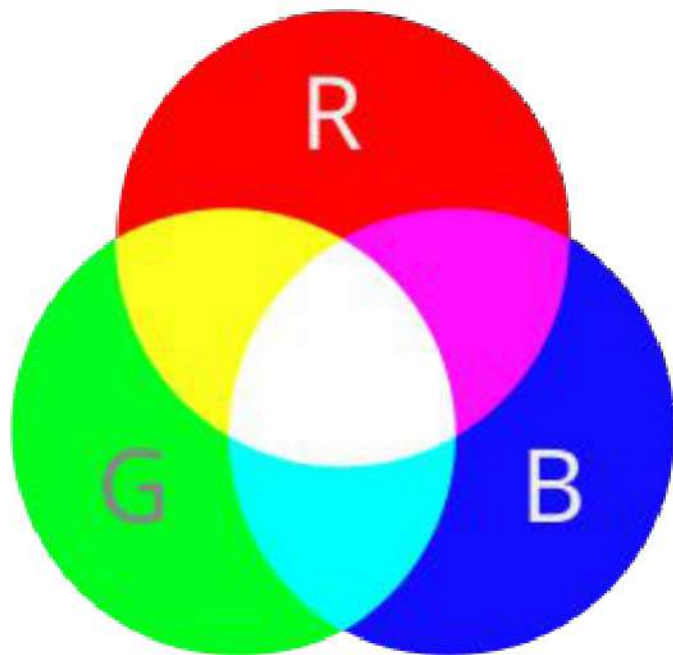
Computer Vision

“컴퓨터가 시각적 정보를 사람처럼 해석하고 이해할 수 있도록 만드는 기술”



Computer Vision

RGB 색공간



Computer Vision

이미지 = “배열 (Array)”

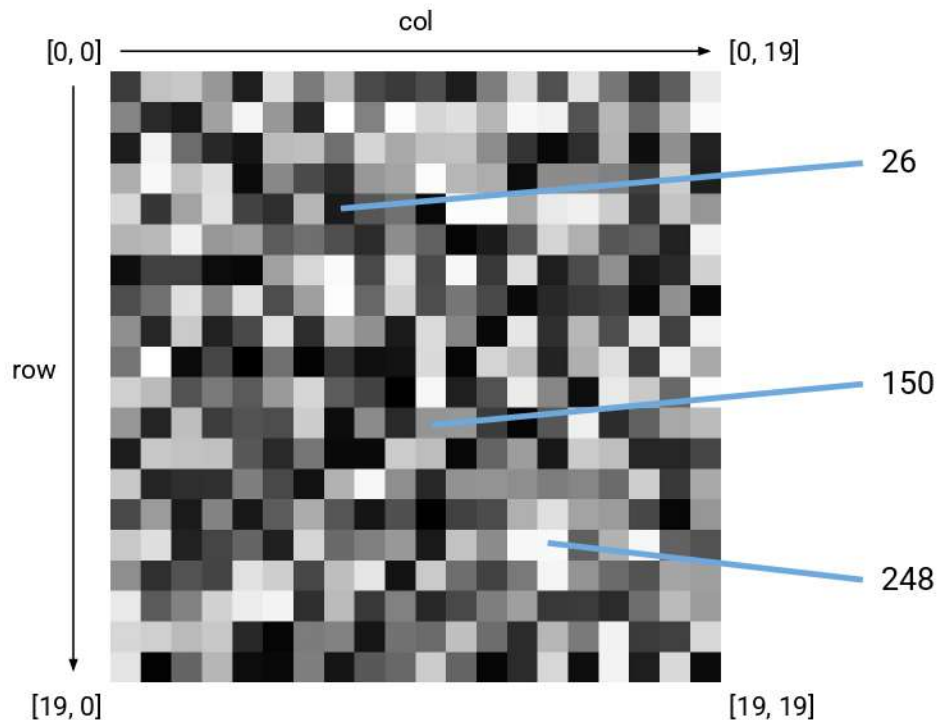
row = 행 (가로 줄)

column = 열 (세로 줄)

index: [row, col]

각 픽셀 값 = 빛의 세기

uint8 (0~255)

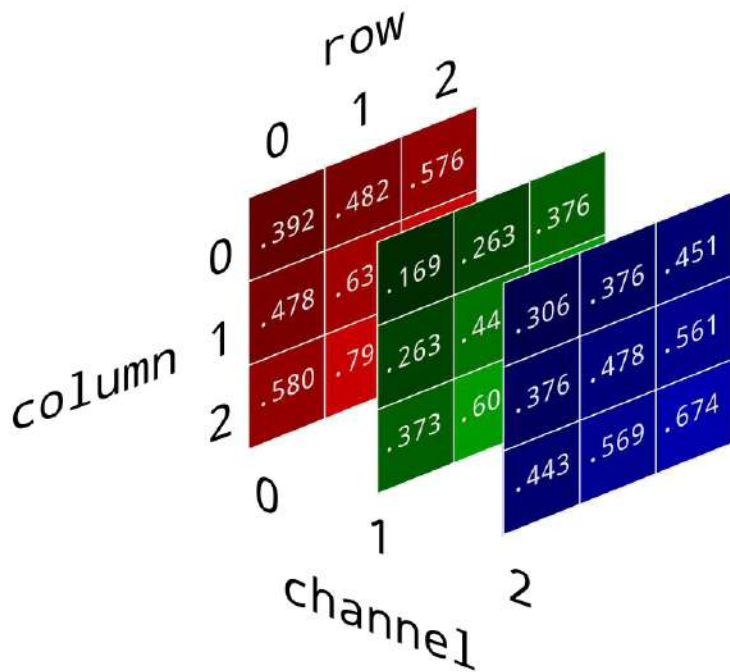


Computer Vision

이미지 = “배열 (Array)”

grayscale: $h * w$

color: $h * w * c$



Computer Vision

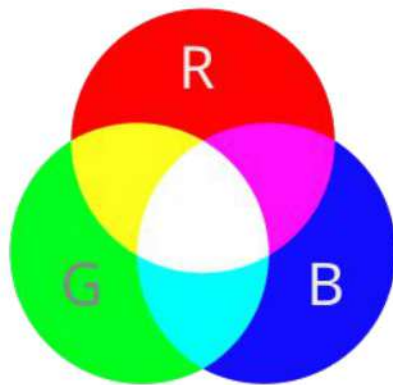
OpenCV:

Computer Vision 분야에서 가장 널리 쓰이는 표준 개발 도구 (오픈소스 라이브러리)

Computer Vision

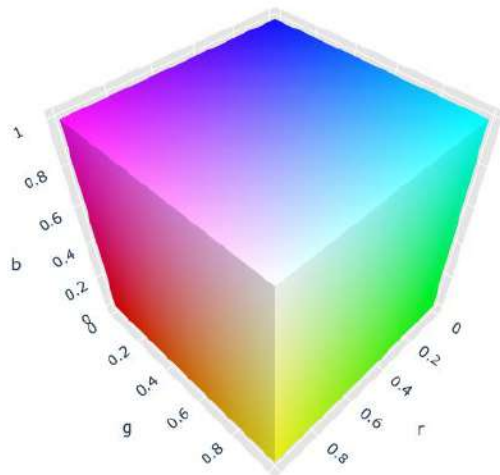
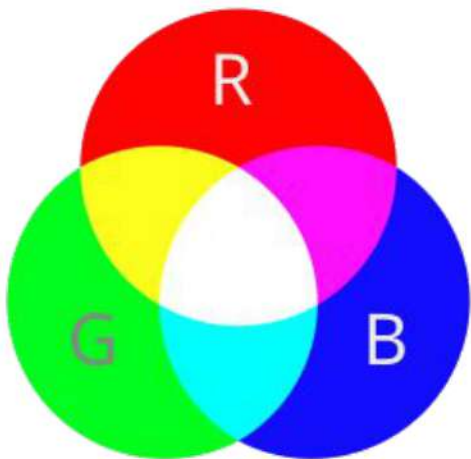
색공간 (Color Space):

- 색을 일정한 기준으로 표현하는 방식
- 색공간마다 표현 가능한 색의 범위가 다름
- 일관된 색 표현을 위해 색공간 통일
- 색공간 변환을 통해 이미지 전처리 효율을 극대화

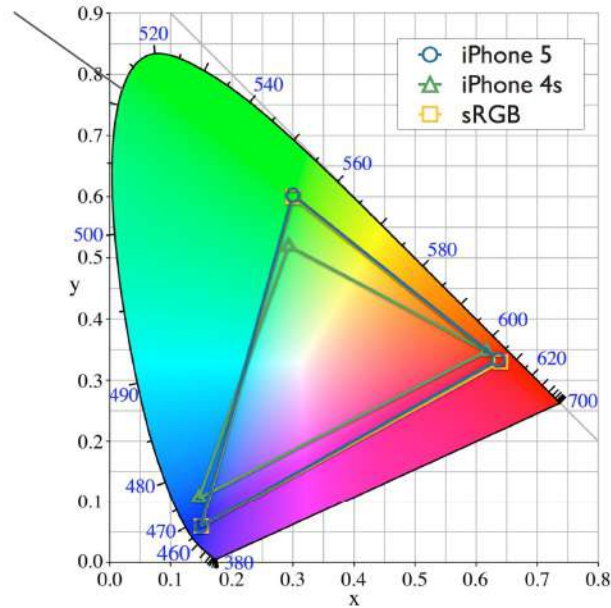


Computer Vision

RGB/BGR



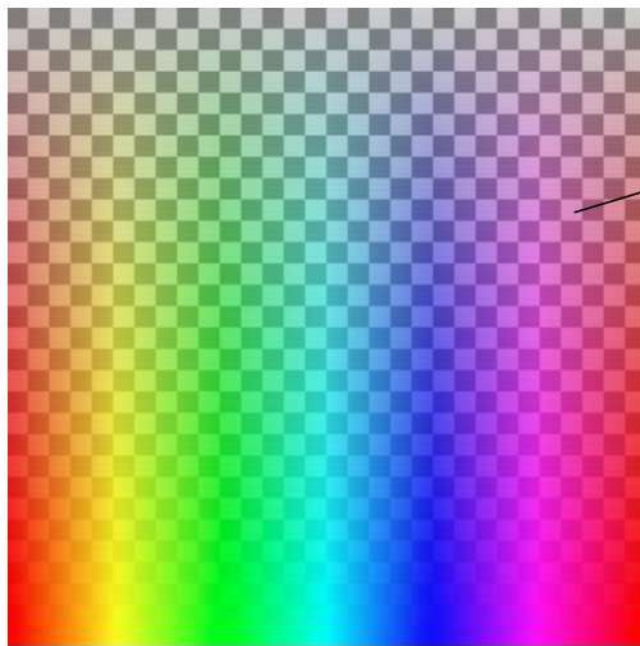
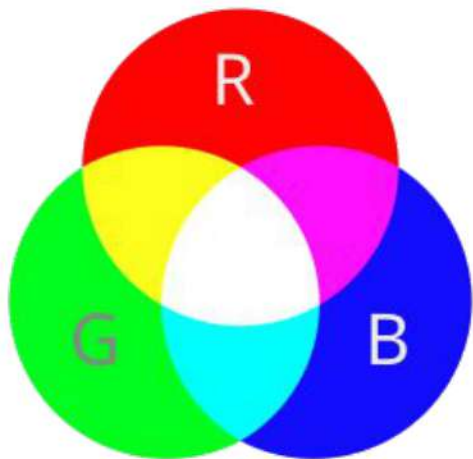
CIE 1931 색공간
(가시광선 영역)



Color Gamut (색 영역)

Computer Vision

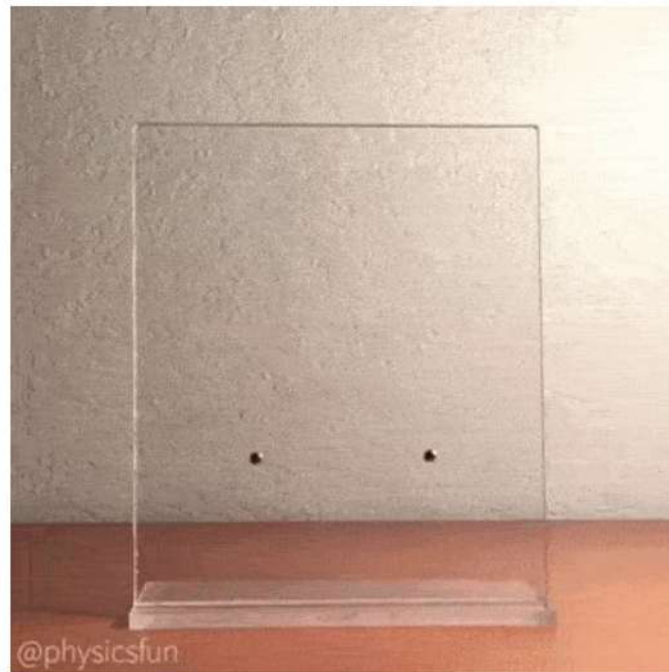
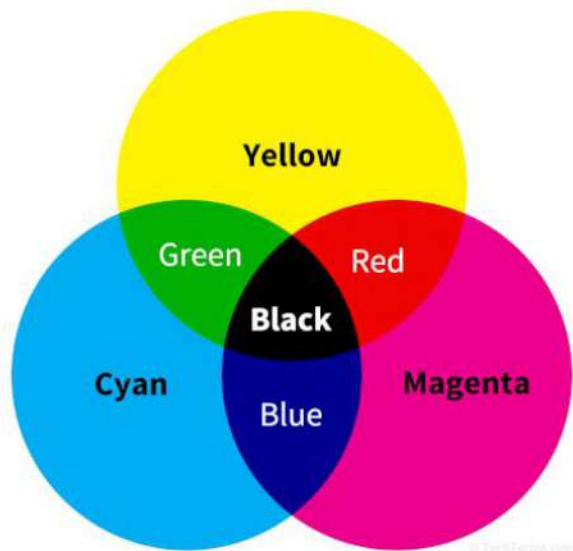
RGBA



$h * w * 4$
Channel: r, g, b, a

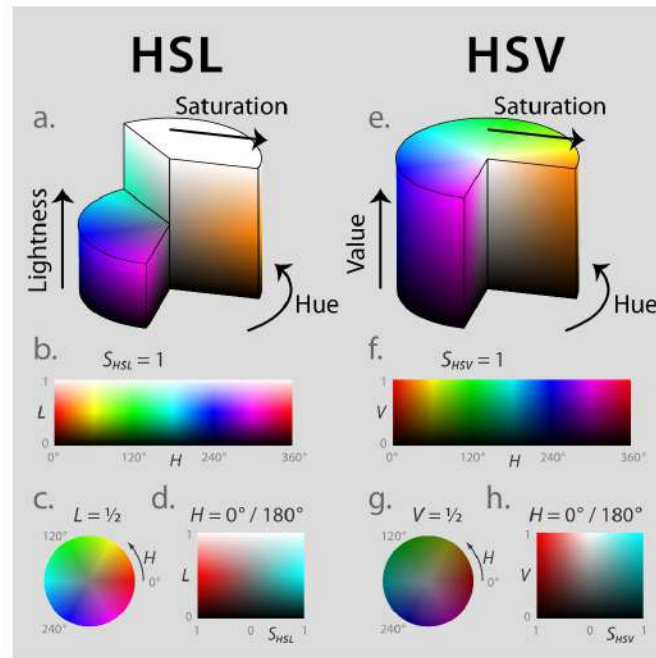
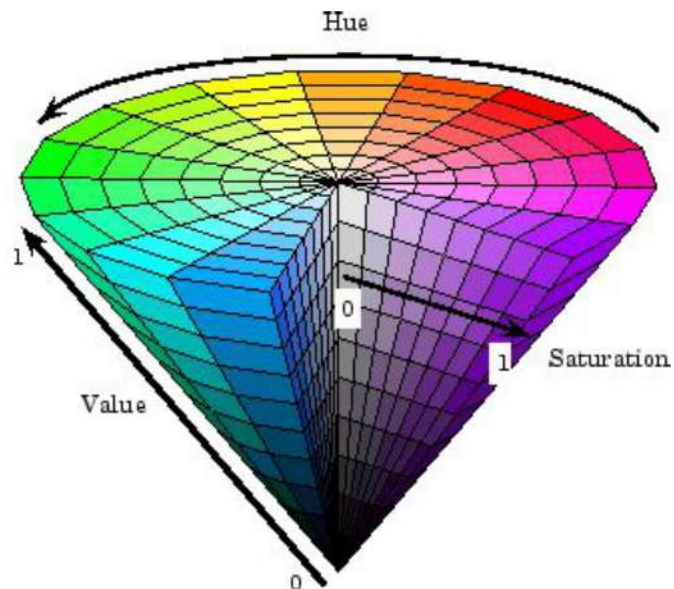
Computer Vision

CMYK



Computer Vision

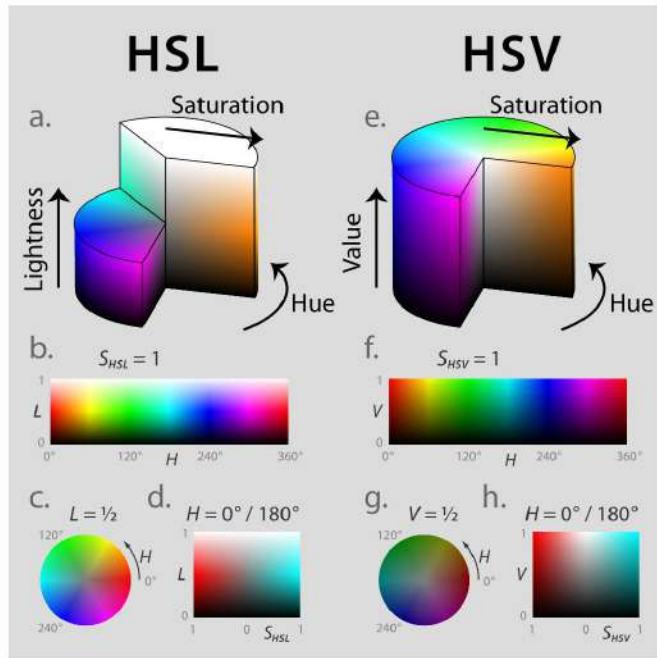
HSV/HSL



Computer Vision

HSV

- Hue(색상), Saturation(채도), Value(명도)
- 색상과 밝기 정도를 분리하여 표현
- 조명 변화에 강한 색상 검출
- 객체 인식 및 이미지 전처리에 유용



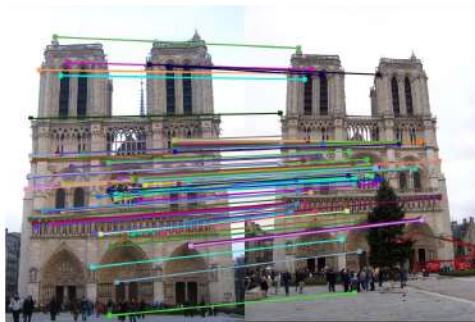
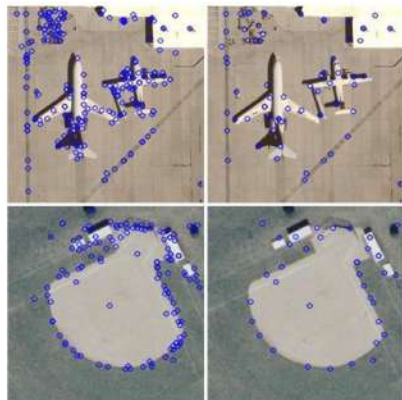
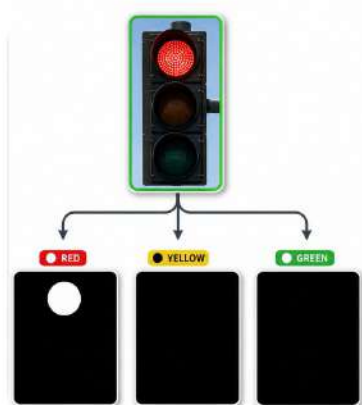
Computer Vision

객체 탐지 (Object Detection):

구분	Rule-based	Learning-based
특징 정의	사람이 직접 정의	데이터로부터 자동 학습
학습 데이터	불필요	필요
환경 변화 대응	낮음	상대적으로 높음
구현 난이도	단순한 문제에서는 쉬움	학습 과정이 필요
복잡한 객체 처리	어려움	상대적으로 유리
대표 기술	HSV Mask, Contour, Haar Cascade	CNN, R-CNN, YOLO

Computer Vision

객체의 특징 추출



Computer Vision

색상 기반 분할 (Color Segmentation):

- 기법: 색상 기반 분할 (Color Segmentation)
- 설명: 원하는 특정 색상만 분리/추출
- 방법: 색상 임계값 처리 (Color Thresholding)
- 결과: 이진 마스크 (Binary Mask)

Computer Vision

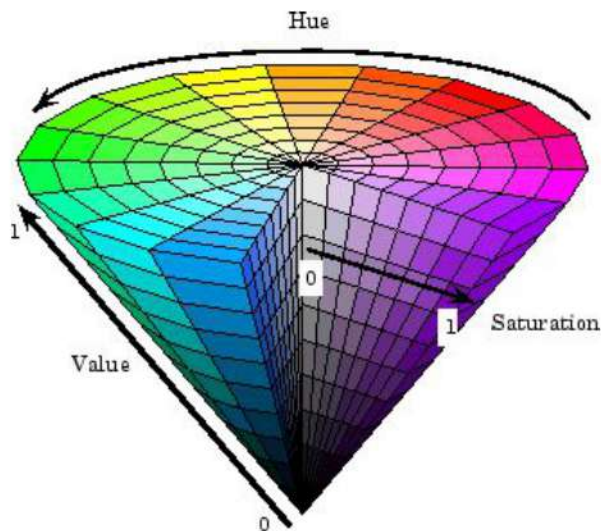
이진 마스크 (Binary Mask):

- Color Thresholding (색상 범위 검출)의 결과
- 오직 두가지 색상 (흰색/검정색)으로 표현하는 “마스크”
- 원하는 색은 255 (흰색), 원치 않는 색은 0 (검정색)으로 변환

Computer Vision

색공간 제약:

- RGB 색공간의 제약
 - 밝기 변화에 RGB 값이 급격히 변함
- HSV로 변환
 - H (Hue): 색의 종류
 - S (Saturation): 색의 채도
 - V (Value): 빛의 밝기 → 조명 오염 무시 & 순수 색상 정보 (H) 사용 가능



Computer Vision

외곽선 (Contour):

```
contours, hierarchy = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

- 검출 모드
 - `cv2.RETR_EXTERNAL`: 구멍 뚫린 객체의 바깥쪽 테두리만 검출
- 근사화 방법:
 - `cv2.CHAIN_APPROX_SIMPLE`: 직선 경로 위에 있는 픽셀 좌표 전체 대신 꼭짓점 좌표만 저장

Computer Vision

외곽선 (컨투어, Contour):

```
contour = max(contours, key=cv2.contourArea)
```

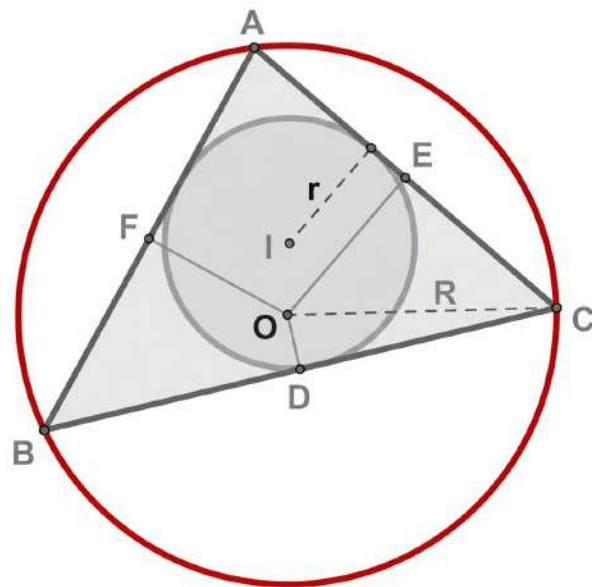
- Key (선택 조건)
 - `cv2.contourArea`: 가장 넓은 면적의 윤곽선 선택
 - `cv2.arcLength`: 가장 긴 둘레의 윤곽선 선택

Computer Vision

최소 외접원 (Minimum Enclosing Circle):

“외곽선의 모든 점들을 포함하는 가장 작은 원”

(x, y) , $radius = cv2.minEnclosingCircle(contour)$



Computer Vision

무게중심 (Centroid):

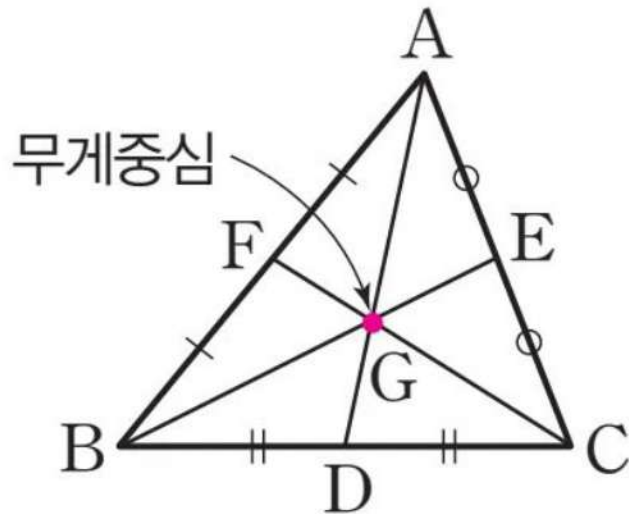
“객체(Contour)의 픽셀 분포를 기반으로 계산한
기하학적 중심점(객체의 중심 좌표)”

무게중심 = 픽셀 가중치 합 / 전체 면적

```
M = cv2.moments(contour)
```

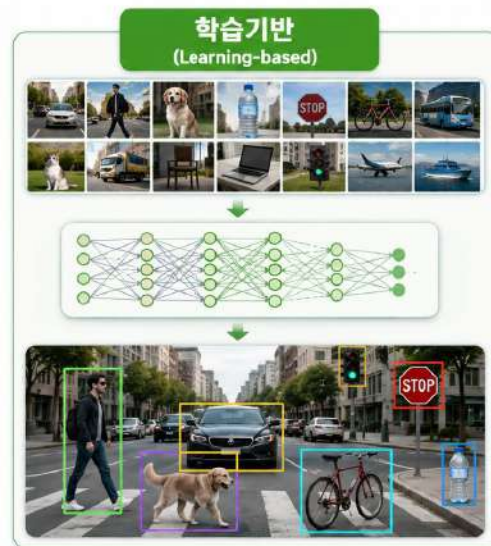
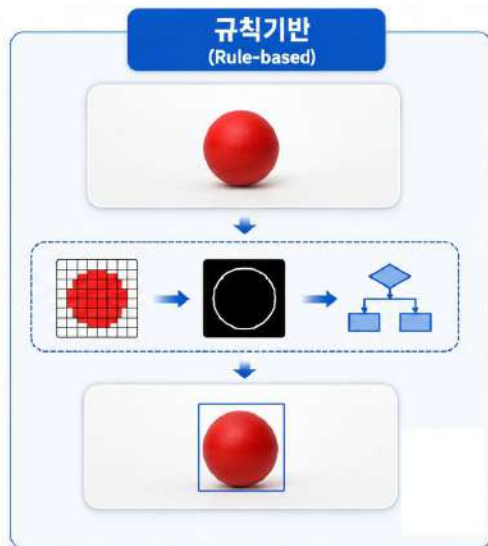
```
cX = int(M["m10"] / M["m00"])
```

```
cY = int(M["m01"] / M["m00"])
```



Computer Vision

객체 탐지 (Object Detection):



Computer Vision



Computer Vision

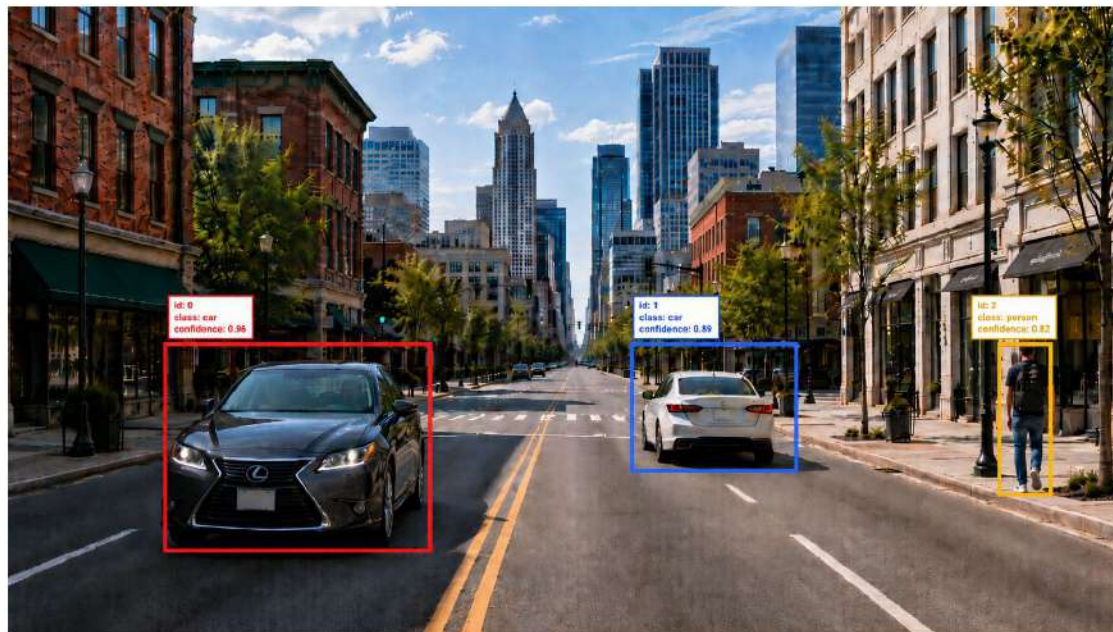
분류(Classification) vs. 탐지(Detection):

Predicted Class: Car



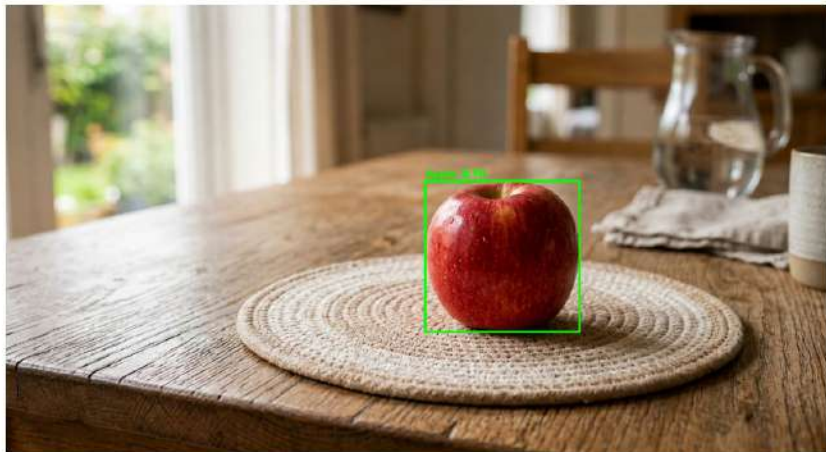
Computer Vision

분류(Classification) vs. 탐지(Detection):



Computer Vision

학습 기반 객체 탐지



어디에 있어? → **Bounding Box**

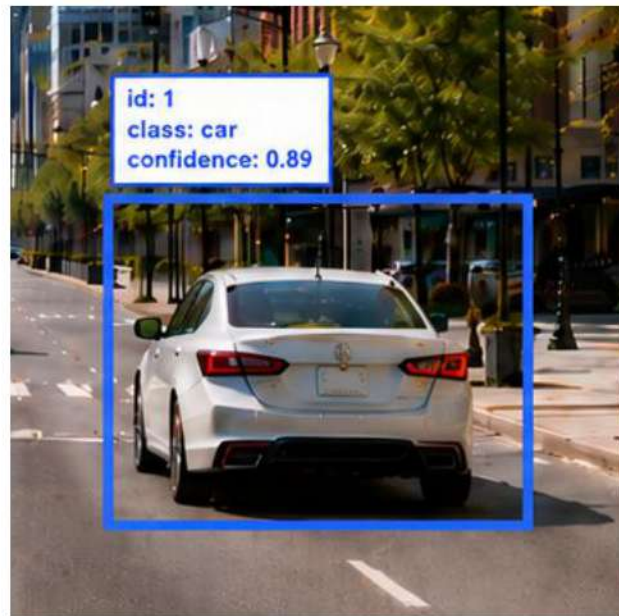
뭔데? → **Class**

얼마나 확신? → **Confidence**

Computer Vision

Bounding Box (BBox):

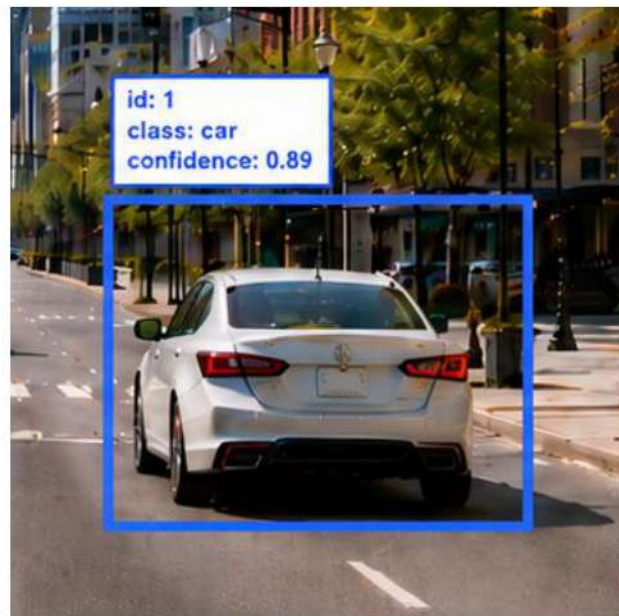
- 탐지된 객체를 둘러싸는 직사각형 영역
- OpenCV 이미지 좌표계:
 - 좌측 상단이 원점 (0, 0)
 - 오른쪽: x 증가
 - 왼쪽: y 증가



Computer Vision

Bounding Box (BBBox) 표현 방식:

1. $(x1, y1, x2, y2)$
2. (x, y, w, h)
3. (cx, cy, w, h)
4. $(cx_norm, cy_norm, w_norm, h_norm)$



Computer Vision

Two-stage Detector (2단계 검출기) vs. One-stage Detector (1단계 검출기):

구분	Two-stage	One-stage
탐지 과정	후보 영역 생성 후 분류	한 번에 직접 예측
후보 영역	명시적으로 생성	별도 후보 생성 단계 없음
처리 구조	복잡함	상대적으로 단순함
추론 속도	상대적으로 느린 경향	상대적으로 빠른 경향
작은 객체·정밀 탐지	유리한 경우가 많음	모델 구조에 따라 달라짐
실시간 처리	상대적으로 불리	유리
대표 모델	Faster R-CNN	YOLO

Computer Vision

YOLO (You Only Look Once):



Computer Vision

YOLO (You Only Look Once):

- CNN 기반 객체 탐지 모델
- One-stage Detector
- 이미지 전체를 신경망에 한 번 입력하여, 이미지 속 객체들을 전부 탐지
- 객체들의 정보 동시 예측
 - 위치: Bounding Box
 - 종류: Class
 - 신뢰도: Confidence

Computer Vision

YOLO (You Only Look Once):

- 실시간 처리에 적합
 - 별도의 Region Proposal 단계가 없음
 - Feature Map을 여러 객체가 공유함
 - 여러 위치의 예측을 병렬로 계산함
 - GPU의 병렬 연산을 효율적으로 활용
 - 작은 모델부터 큰 모델까지 선택 가능

Computer Vision

COCO 데이터셋:

Common Objects in Context

ID	Class	의미
0	person	사람
1	bicycle	자전거
2	car	자동차
3	motorcycle	오토바이
4	airplane	비행기
5	bus	버스
6	train	기차
7	truck	트럭
8	boat	보트
9	traffic light	신호등
10	fire hydrant	소화전
11	stop sign	정지 표지판
12	parking meter	주차 미터기
13	bench	벤치
14	bird	새
15	cat	고양이
16	dog	개
17	horse	말
18	sheep	양
19	cow	소

ID	Class	의미
20	elephant	코끼리
21	bear	곰
22	zebra	얼룩말
23	giraffe	기린
24	backpack	배낭
25	umbrella	우산
26	handbag	핸드백
27	tie	넥타이
28	suitcase	여행 가방
29	frisbee	원반
30	skis	스키
31	snowboard	스노보드
32	sports ball	스포츠 공
33	kite	연
34	baseball bat	야구 방망이
35	baseball glove	야구 글러브
36	skateboard	스케이트보드
37	surfboard	서프보드
38	tennis racket	테니스 라켓
39	bottle	병

ID	Class	의미
40	wine glass	와인잔
41	cup	컵
42	fork	포크
43	knife	나이프
44	spoon	숟가락
45	bowl	그릇
46	banana	바나나
47	apple	사과
48	sandwich	샌드위치
49	orange	오렌지
50	broccoli	브로콜리
51	carrot	당근
52	hot dog	핫도그
53	pizza	피자
54	donut	도넛
55	cake	케이크
56	chair	의자
57	couch	소파
58	potted plant	화분
59	bed	침대

ID	Class	의미
60	dining table	식탁
61	toilet	변기
62	tv	TV
63	laptop	노트북
64	mouse	마우스
65	remote	리모컨
66	keyboard	키보드
67	cell phone	휴대전화
68	microwave	전자레인지
69	oven	오븐
70	toaster	토스터
71	sink	싱크대
72	refrigerator	냉장고
73	book	책
74	clock	시계
75	vase	꽃병
76	scissors	가위
77	teddy bear	곰 인형
78	hair drier	헤어드라이어
79	toothbrush	칫솔

Computer Vision

COCO 데이터셋:

Common Objects in Context

항목	내용
이미지 내용	일상 장면 속 다양한 객체의 컬러 이미지
이미지 크기	이미지마다 다름
채널	RGB 3채널
클래스	80개
전체 이미지	약 330,000장
객체 인스턴스	약 1,500,000개
주요 용도	객체 탐지·분할·키폰트·캡셔닝



SLAM

Simultaneous Localization And Mapping



위치 추정
(Localization)



지도 작성
(Mapping)



SLAM

SLAM input/output

Input:

- 카메라 이미지
- LiDAR laser scan
- IMU orientation
- 바퀴 encoder
- GNSS

Output:

- Map (2D map, Point Cloud, Mesh, Topology)
- Robot pose (x, y, z, w)

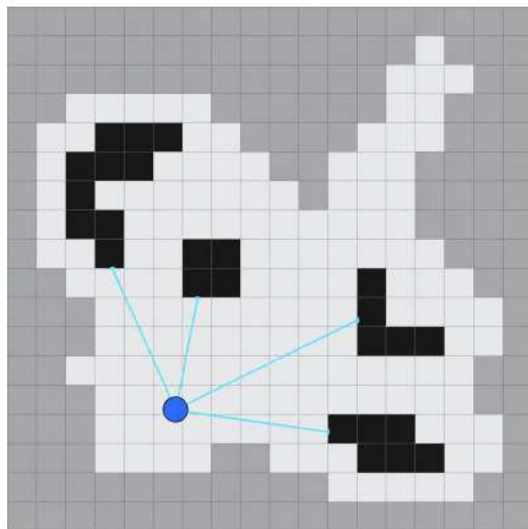
SLAM: Mapping

Occupancy Grid Map: 지도를 grid로 나눠 각 칸(cell) 정의

- Free (빈공간)
- Occupied (장애물)
- Unknown (미관측 / 데이터 부족)

Occupancy Grid의 Cell:

- 현실: 거리 (m)
- 지도: 픽셀



SLAM: Mapping

각 cell의 occupancy 계산 (0=free, 1=occupied)

Bayesian Filter:

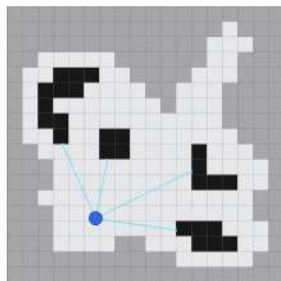
$$P(c_i = occ \mid z_{1:t}, x_{1:t}) \qquad P(c_i = occ)_{t=0} = 0.5$$

$$P(c_i = occ \mid z_{1:t}, x_{1:t}) = \frac{P(z_t \mid c_i = occ) * P(c_i = occ \mid z_{1:t-1})}{P(z_t \mid z_{1:t-1})}$$

$$= P(occ \mid hit) = \frac{P(hit \mid occ) * P(occ \mid prev_z)}{P(hit \mid prev_z)}$$

SLAM: Mapping

Occupancy 데이터 관리:

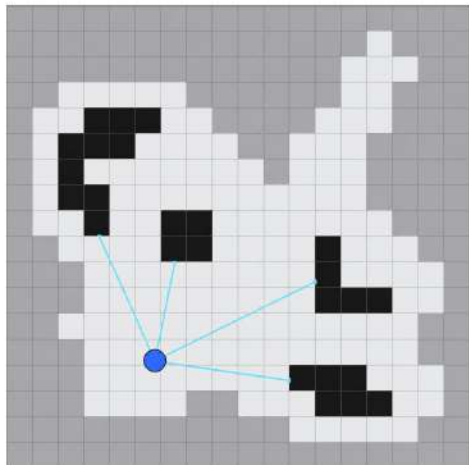


- 확률값 보존 (0~1)
 - 원본 정보 직관적인 보존
 - 고정밀도 실수의 데이터 부담
 - Underflow 발생 위험
- 정수형 Scaling (0~100)
 - 효율적인 데이터 관리 및 메모리 절감
 - 연산 단순화
 - 정밀도 손실
- Log Space 변환 ($-\infty \sim 0$)
 - Underflow 방지
 - 작은 확률을 안정적으로 표현
 - Throughput 향상 가능
- Trinary 표현 (0/100/-1)
 - 센서 노이즈 제거 가능 (강제 영역 분리)
 - 고속 판별 가능 (판단 위주)
 - 정보 유실 (분석/디버깅 불가능)

SLAM: Localization

Localization 개념:

“로봇이 미리 만들어진 지도 위에서 자신의 위치 (x, y, θ) 를 추정하는 문제”



SLAM: Localization

Measurement-only Estimation

Inertial Measurement Unit, IMU

- Accelerometer : 선형 가속도
- Gyroscope : 각속도
- Magnetometer : 절대 방위(방향)



SLAM: Localization

Measurement-only Estimation

Wheel Encoder

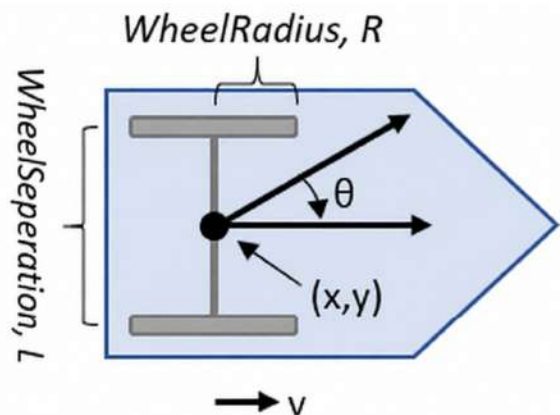
- Encoder Pulse Count
 - 바퀴 이동 거리
 - 로봇 이동 거리
 - 로봇 회전 각도



SLAM: Localization

Measurement-only Estimation

Differential Drive Robot (차동 구동 로봇)의 Wheel Encoder



바퀴 이동 거리

$$d_l = \frac{C_l}{PPR} \times 2\pi R$$

$$d_r = \frac{C_r}{PPR} \times 2\pi R$$

d : 바퀴 이동 거리

C : Pulse Count

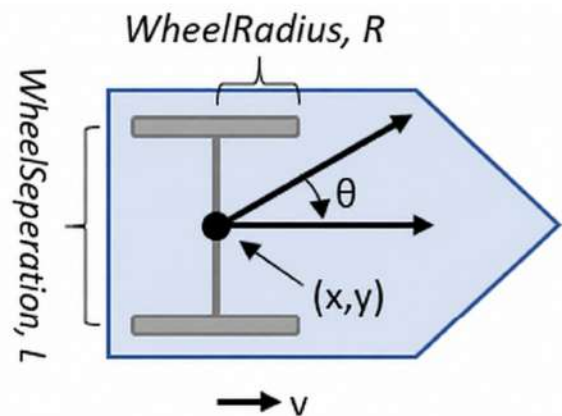
PPR : Pulse Per Revolution

R : 반지름

SLAM: Localization

Measurement-only Estimation

Differential Drive Robot (차동 구동 로봇)의 Wheel Encoder

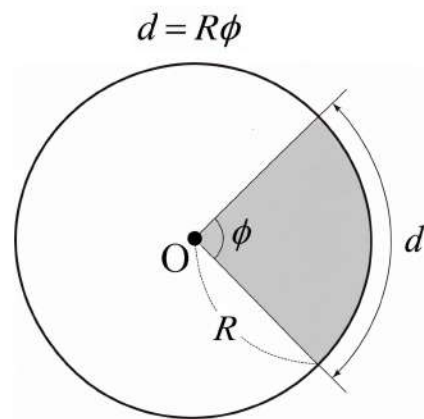


바퀴 이동 거리

$$d_l = R\phi_l$$

$$d_r = R\phi_r$$

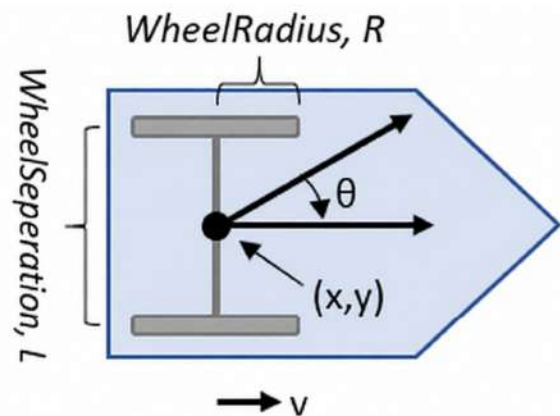
ϕ : 바퀴 회전각



SLAM: Localization

Measurement-only Estimation

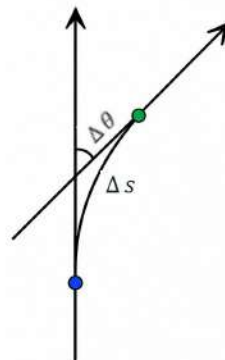
Differential Drive Robot (차동 구동 로봇)의 Wheel Encoder



로봇 이동 거리 · 회전 각도

$$\Delta s = \frac{d_r + d_l}{2}$$

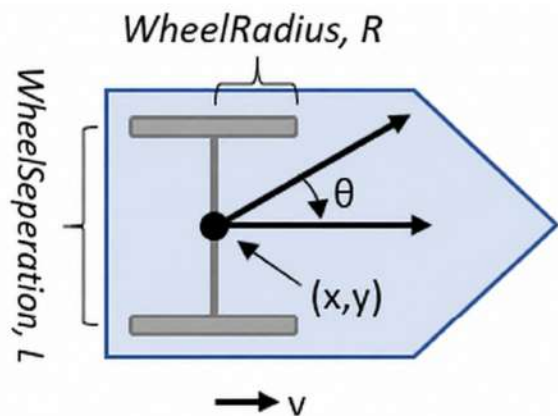
$$\Delta \theta = \frac{d_r - d_l}{L}$$



SLAM: Localization

Measurement-only Estimation

Differential Drive Robot (차동 구동 로봇)의 Wheel Encoder

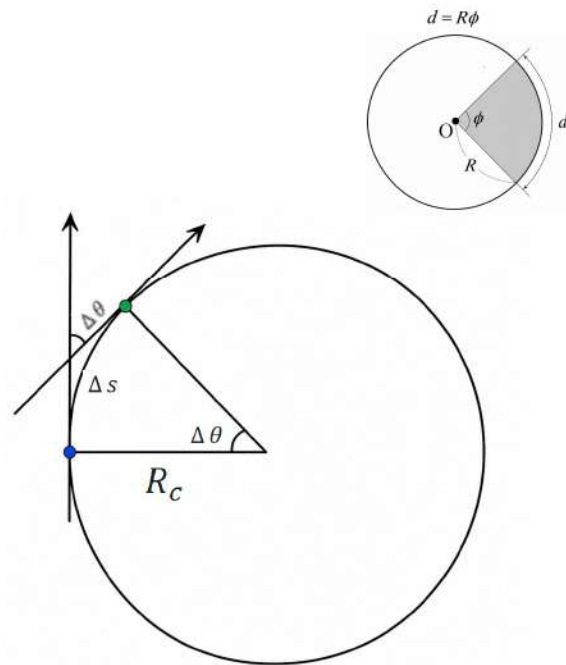


로봇 최종 위치

$$x = R_c \sin(\Delta \theta)$$

$$y = R_c (1 - \cos(\Delta \theta))$$

$$R_c = \frac{\Delta s}{\Delta \theta}$$

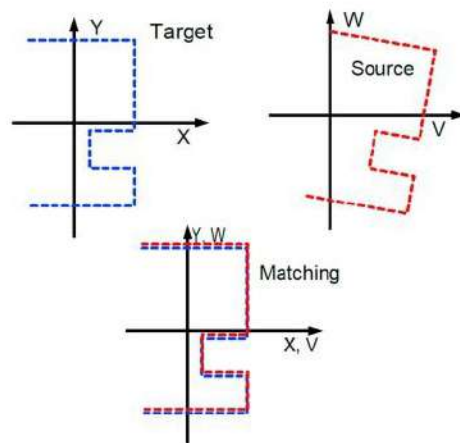


SLAM: Localization

Measurement-only Estimation

Scan Matching (Scan-to-Scan Matching, Scan-to-Map Matching)

- 현재와 과거 laser scan 비교 (LiDAR Odometry)
- 현재 laser scan과 지도 비교 (Localization)

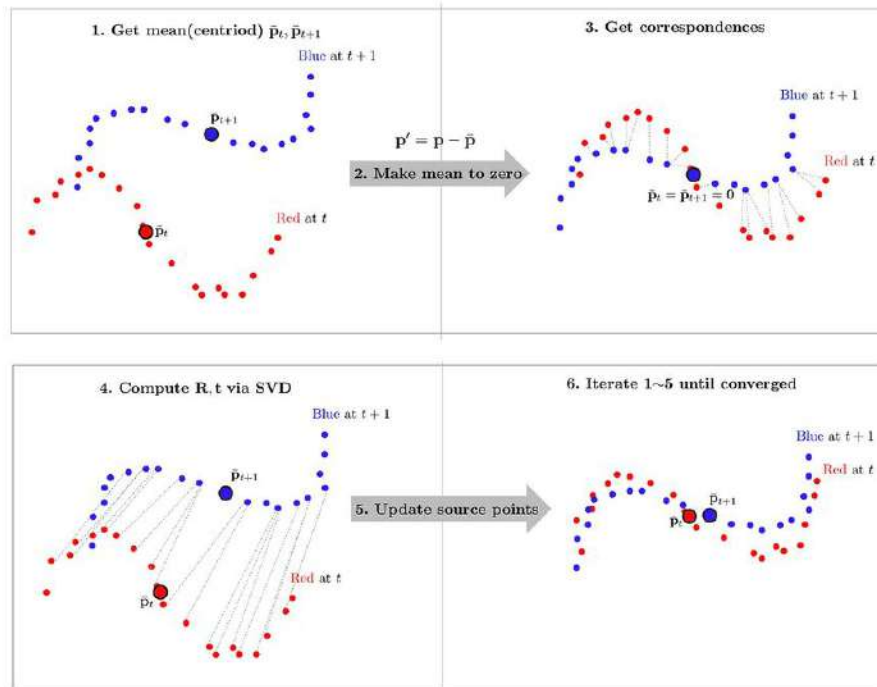


SLAM: Localization

Measurement-only Estimation

Iterative Closest Point, ICP

- 두 Point Cloud 간 위치 정합
- 각 Point의 Closest Point 대응 관계 탐색
- Rotation & Translation 반복 계산
- 오차 최소화/수렴 시 종료

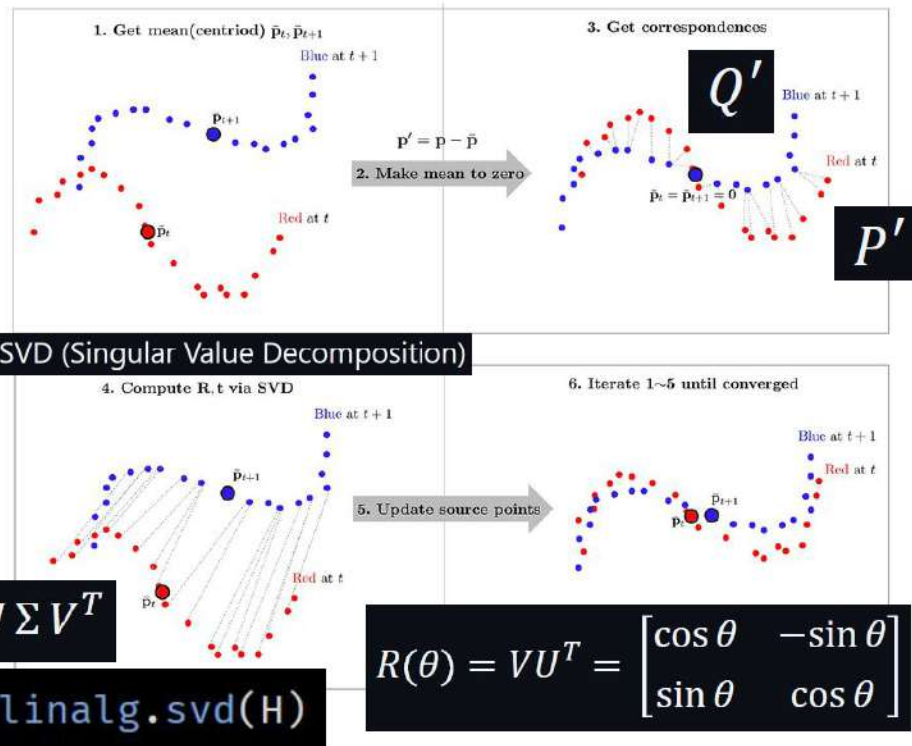


SLAM: Localization

Measurement-only Estimation

Iterative Closest Point, ICP

- 두 Point Cloud 간 위치 정합
- 각 Point의 Closest Point 대응 관계 탐색
- Rotation & Translation 반복 계산
- 오차 최소화/수렴 시 종료

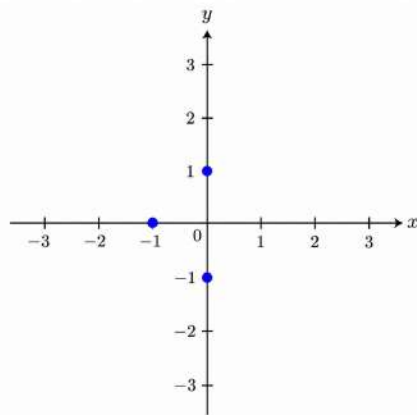
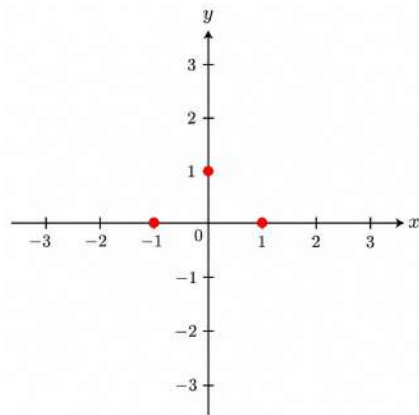


SLAM: Localization

Measurement-only Estimation

Iterative Closest Point, ICP

$$P = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & 0 \end{bmatrix}, \quad Q = \begin{bmatrix} 0 & 1 \\ -1 & 0 \\ 0 & -1 \end{bmatrix}$$

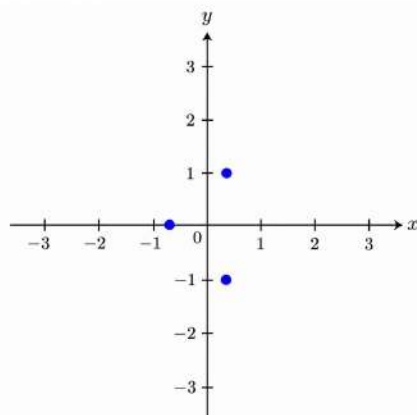
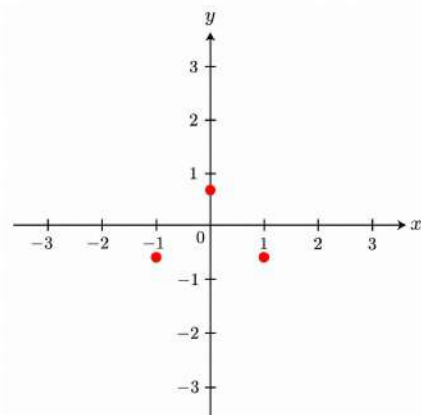


SLAM: Localization

Measurement-only Estimation

Iterative Closest Point, ICP

$$P' = \begin{bmatrix} 1 & -1/3 \\ 0 & 2/3 \\ -1 & -1/3 \end{bmatrix}, \quad Q' = \begin{bmatrix} 1/3 & 1 \\ -2/3 & 0 \\ 1/3 & -1 \end{bmatrix}$$



SLAM: Localization

Measurement-only Estimation

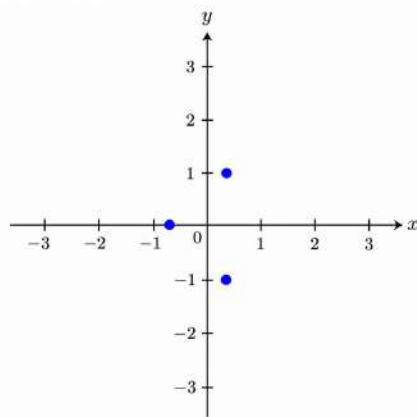
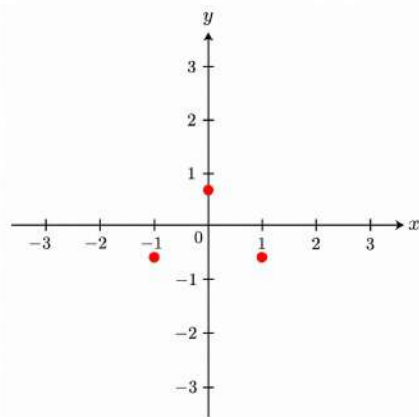
Iterative Closest Point, ICP

$$H = P'^T Q' = \begin{bmatrix} 0 & 2 \\ -1 & 0 \end{bmatrix} = U \Sigma V^T$$

$$U = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad V = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$R(\theta) = VU^T = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

$$\theta = 90^\circ$$

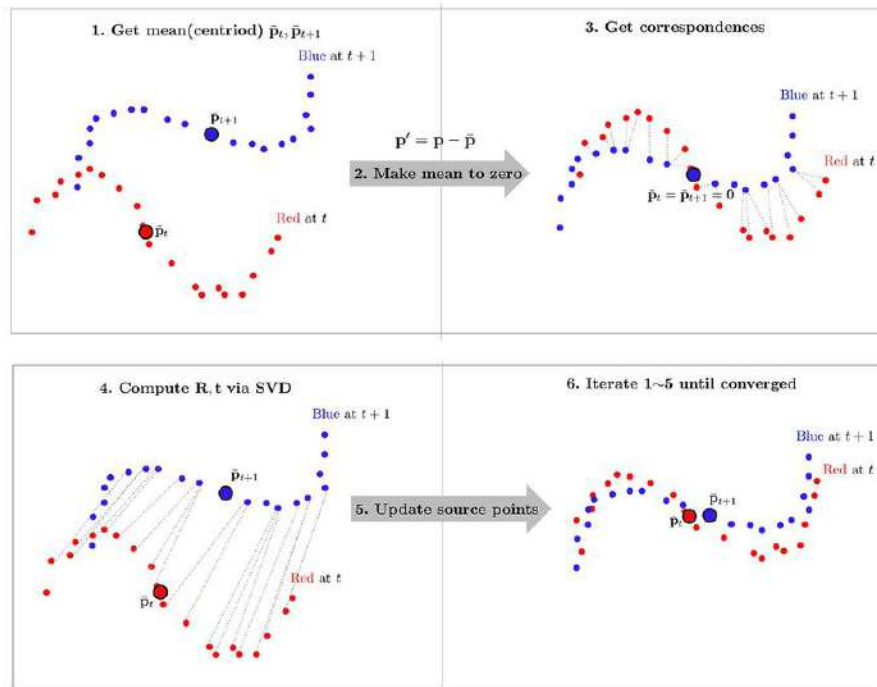


SLAM: Localization

Measurement-only Estimation

Iterative Closest Point, ICP

- 두 Point Cloud 간 위치 정합
- 각 Point의 Closest Point 대응 관계 탐색
- Rotation & Translation 반복 계산
- 오차 최소화/수렴 시 종료

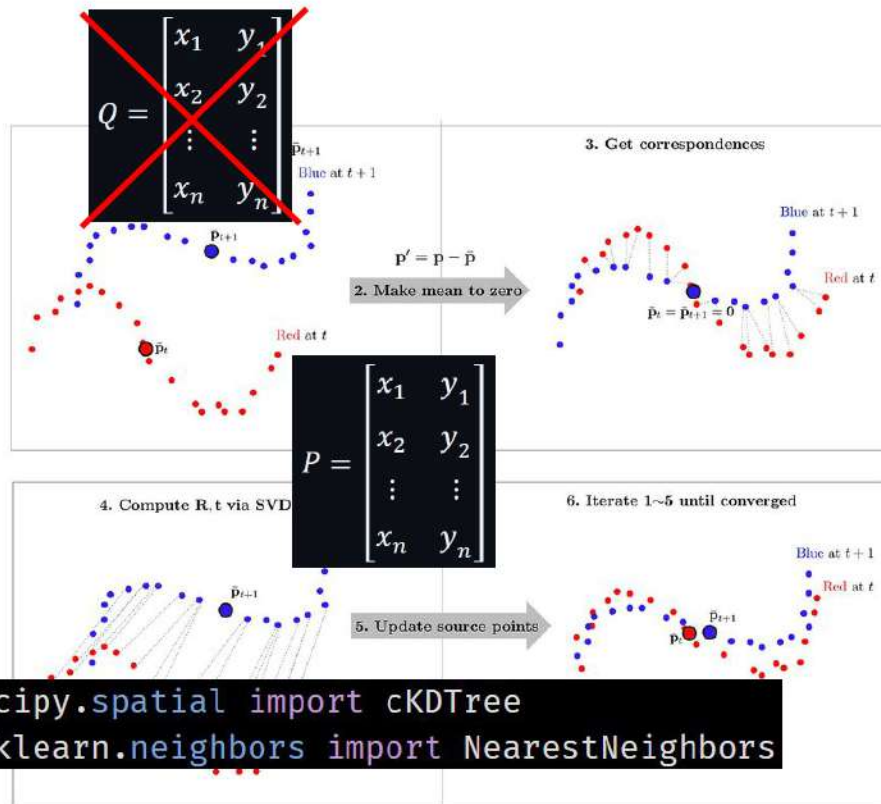


SLAM: Localization

Measurement-only Estimation

Iterative Closest Point, ICP

- 두 Point Cloud 간 위치 정합
- 각 Point의 Closest Point 대응 관계 탐색
- Rotation & Translation 반복 계산
- 오차 최소화/수렴 시 종료

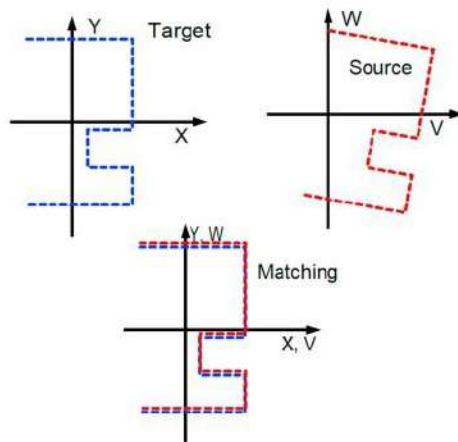


SLAM: Localization

Measurement-only Estimation

Scan Matching (Scan-to-Scan Matching, Scan-to-Map Matching)

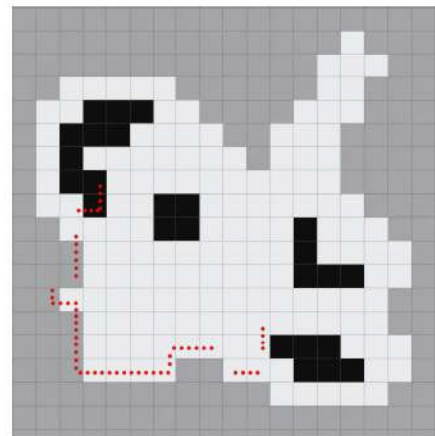
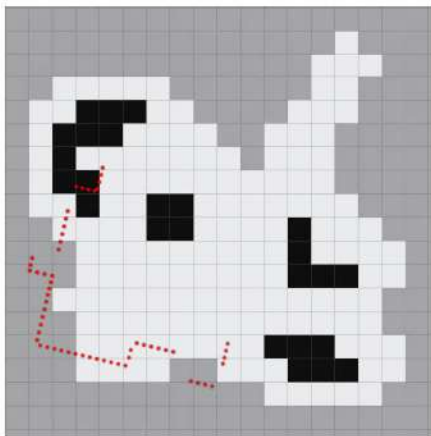
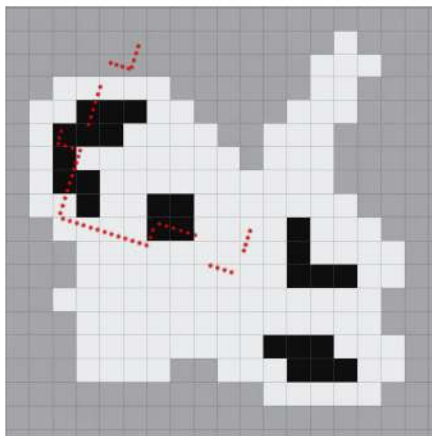
- 현재와 과거 laser scan 비교 (LiDAR Odometry)
- 현재 **laser scan**과 지도 비교 (**Localization**)



SLAM: Localization

Measurement-only Estimation

Scan-to-Map Matching



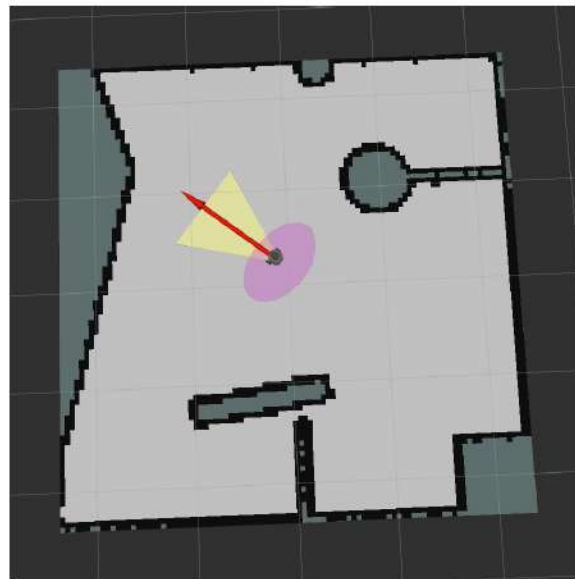
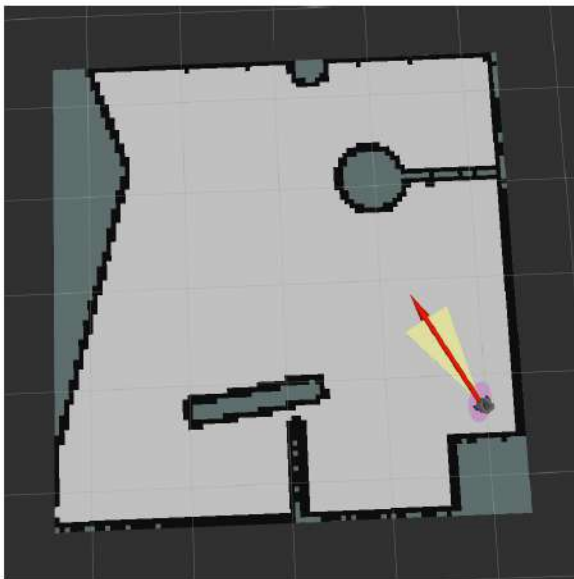
SLAM: Localization

시뮬레이션 주요 전제:

- **Kidnapped Robot Problem** 다루지 않음
 - 이동의 연속성
- **Initial Position & Orientation** 아는 상태로 시작
 - Odometry만으로도 위치 추정 가능

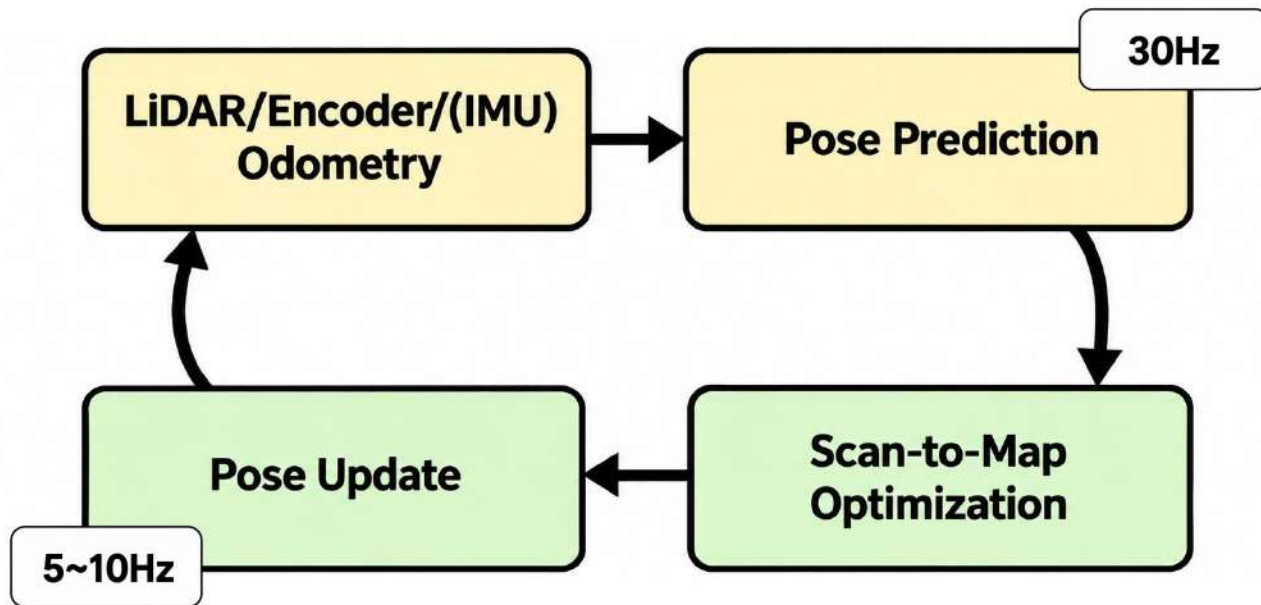
SLAM: Localization

Prediction & Update Cycle



SLAM: Localization

Prediction & Update Cycle



SLAM: Localization

Bayesian Estimation

Kalman Filter

```
1:  Algorithm Kalman_filter( $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$ ):  
2:     $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$   
3:     $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$   
4:     $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$   
5:     $\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$   
6:     $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$   
7:    return  $\mu_t, \Sigma_t$ 
```

μ : state estimate

Σ : estimate covariance matrix

u : control

z : measurement

A : state transition model

B : control input model

R : process noise covariance matrix

K : Kalman gain

C : observation model

SLAM: Localization

Bayesian Estimation

Kalman Filter 이점:

- 과거 **state**를 보기에 높은 정확도
- 센서 노이즈 배제
 - 움직임 예측 + 측정값으로 보정 (센서 측정에만 의존 X)
 - 추정 위치를 **Gaussian** 분산
- 센서 측정값 없이 예측 가능 (고장/유실/불량)
- CV tracking에 유리

SLAM: Localization

Bayesian Estimation

Kalman Filter 단점:

- 초기값 없으면 매우 불안정
- 선형/베이지안 상태 추정 문제에 국한
- 1개 위치만 예측

SLAM: Localization

Bayesian Estimation

Deterministic vs. Stochastic Method

Deterministic (결정론적)

- 1개 위치만 예측
- 유사한/대칭 구조에 취약

Stochastic (확률론적)

- 확률 모델(분포) 사용하여 **pose** 추론
- 유사한/대칭 구조들에 가능성 열어둠

SLAM: Localization

Bayesian Estimation

Monte Carlo 추정법

- 랜덤 샘플을 많이 뽑아서 확률을 근사하는 방법
- 랜덤 샘플이 많을수록 실제 값으로 수렴하는 성질
- 오차 존재 (샘플이 많을 수록 오차 줄어듦 $1/\sqrt{n}$)

SLAM: Localization

Bayesian Estimation

Monte Carlo 추정법 : Particle Filter

- Monte Carlo의 샘플 = Particle
- Particle
 - = 지도에 흩뿌려놓은 점
 - = 로봇의 가능한 위치들

SLAM: Localization

Bayesian Estimation

Particle Filter 4-step 과정:

1. Initialization: particle 초기 분산
2. Prediction: 로봇 이동 후 particle을 로봇 이동 경로에 맞춰 같이 이동
3. Update: 스캔 결과와 비교 후 각 particle 위치에 로봇이 있을 가능성 계산
4. Resampling: 가능성 큰 particle만 유지

SLAM: Localization

Bayesian Estimation

Initialization Step:

- Particle을 지도 전체에 랜덤 배치
- 각 Particle의 가중치는 $1/n$ 으로 동일

SLAM: Localization

Bayesian Estimation

Prediction Step:

- 로봇 이동
- Particle 이동
- 노이즈 추가
 - Encoder 오차, 바퀴 미끄러짐 등 고려해서 오차를 모델링

SLAM: Localization

Bayesian Estimation

Update Step:

- LiDAR 측정값과 비교
- 실제 scan vs. 지도 기반 expected scan (scan matching)
- 각 Particle 마다 가중치 부여 및 정규화
- $P(\text{sensor} \mid \text{particle})$
 - "이 위치라면 이런 센서 측정값이 나와야 한다"
 - 실제 센서와 비교해서 점수 부여

SLAM: Localization

Bayesian Estimation

Resampling Step:

- 다음 라운드($t+1$)에서 n 개 샘플 뽑기 (중복 가능)
- $w_t^{(i)} \propto$ 다음 t 에서 뽑힐 확률
- 나쁜 (가능성 희박한) Particle 제거됨
- 좋은 (가능성 존재하는) Particle 유지됨

SLAM: Localization

Bayesian Estimation

Resampling Step 문제점:

- 다양성 저하
- 좋은 Particle만 계속 복제
- Local minima에 갇힘
- 여러 가설을 유지해야 하는데, 한 가설만 남는 상태
- PF는 여러 가능성을 동시에 유지하는 것이 핵심

SLAM: Localization

Bayesian Estimation

Resampling Step 해결책:

- 샘플 수 증가
- Jittering: 복제 후 노이즈 추가
- Systematic Resampling: CDF 구간별로 선택

SLAM: Localization

Bayesian Estimation

Resampling Step 해결책:

- Adaptive Resampling: ESS가 낮을때만 Resampling
 - ESS = Effective Sample Size (유효 샘플 수)
 - ESS = “지금 particle들이 실제로 몇 개 정도의 ‘쓸모 있는 샘플’ 역할을 하고 있는가”
 - $ESS \propto$ 정보량
 - Particle의 과도한 제거 방지

SLAM: Localization

Bayesian Estimation

최종 위치 추정:

- MAP (Maximum a Posteriori)
- $x_t = x_t^{(i^*)}, \quad i^* = \operatorname{argmax} w_t^{(i)}$

SLAM: Localization

Bayesian Estimation

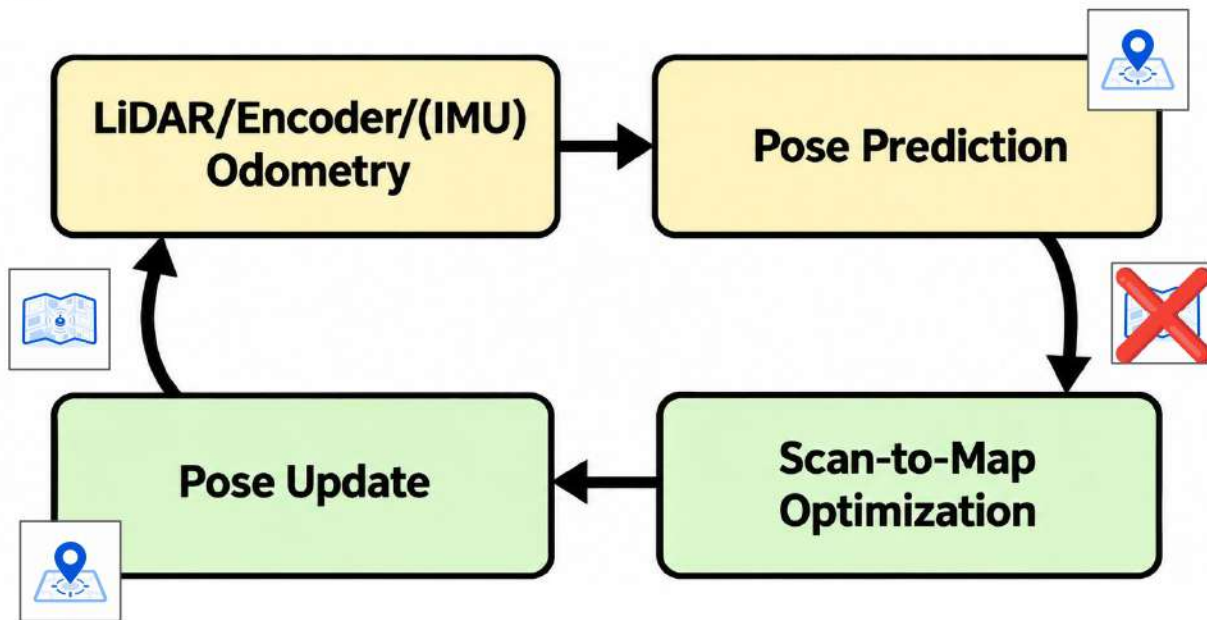
Adaptive Monte Carlo Localization (AMCL)

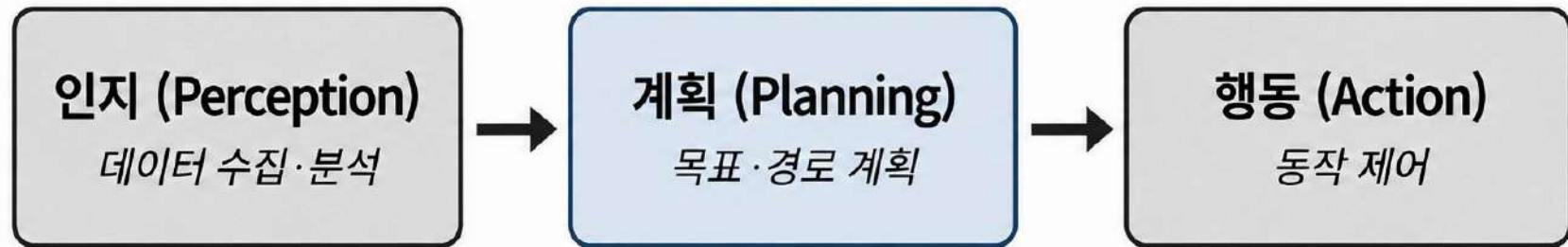
- 고정되지 않은 Particle 수
 - 로봇 위치 확실 → **downsample** (적은 샘플로도 충분히 표현 가능)
 - 로봇 위치 불확실 → 기존 **sample** + 지도 전체에 추가 생성
- 고정되지 않은 Resampling 주기
 - Adaptive Resampling 방식
 - ESS 기반 Resampling

SLAM

Simultaneous Localization And Mapping

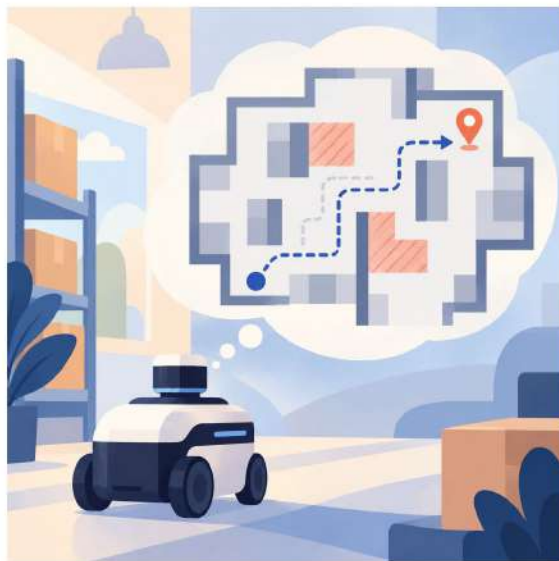
현재 tick에서 둘 다 완료





계획, Planning

- 인지 정보를 기반으로 환경 상황 분석
- 주어진 목표에 따른 행동 계획 수립
- 목적지까지의 최적 경로 생성
- 환경 변화에 따른 계획의 실시간 갱신



계획, Planning

용어 정리

Global Planner :

시작점에서 목표점까지 가기 위해 경로를 계산하기 위한 시스템/모듈

Global Path :

Global Planner가 계산한 시작점에서 목표점까지의 전체적인 이동 경로

Local Planner/Controller :

현재 위치와 주변 환경을 고려하여 가까운 구간에서 움직임을 계산·제어하는 시스템/모듈

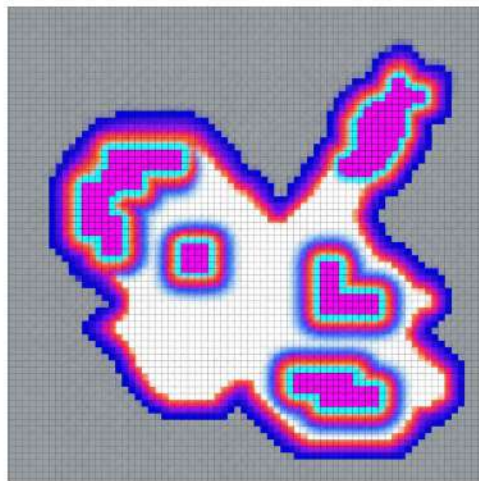
Local Path :

Local Planner/Controller가 현재 움직임을 제어하기 위해 사용하는 가까운 구간의 경로

계획, Planning

Costmap (비용 지도)

안전하게 경로를 계획할 수 있도록 주변 환경 정보를 2차원 격자(grid) 형태로 수치화한 지도



계획, Planning

Global Costmap

안전하게 경로를 계획할 수 있도록 주변 환경 정보를 2차원 격자(grid) 형태로 수치화한 지도

- Occupancy Grid Map은 단순 장애물 여부 (trinary)
- 충돌 여부는 로봇 크기 고려 필수
- 기존 Occupancy Grid Map 위에 안전 거리와 장애물 정보를 층층이 쌓아 올린 지도
- 비용(cost)으로 공간의 위험도 표현
- 각 cell은 1 byte (uint8, 0~255)
- 일반 지도 (trinary) vs. Costmap (0~255)

계획, Planning

Costmap의 Cost Value

- Unknown Area (255):
 - 알 수 없는 영역 (미관측/데이터 부족)
 - Global Planner는 기본적으로 이 영역을 지나가지 않으려고 함
- Occupied Area (254):
 - 해당 픽셀에 닿으면 무조건 충돌하는 치명적 장애물 영역
 - 실제 벽이나 장애물이 위치한 좌표
- Inscribed / Collision Area (253):
 - 내접 장애물 영역 (장애물이 아니지만 충돌하는 픽셀)
 - 로봇의 크기를 고려했을 때 로봇 중심이 진입하면 무조건 외벽과 충돌하게 되는 경계선

계획, Planning

Costmap의 Cost Value

- Traversable / Inflation Area (1 ~ 252):
 - 장애물 주변으로 갈수록 점수가 높아지는 팽창(Inflation) 구역
 - Planner는 이 점수를 바탕으로 벽에서 최대한 멀리 떨어지는 안전한 경로를 계산
- Free Space (0):
 - 자유 이동 영역
 - 장애물이 전혀 없고 안전하게 최고 속도로 달릴 수 있는 공간

계획, Planning

Global Costmap vs. Local Costmap

- Global Costmap:
 - 지도 전체 반영
 - Map Frame 기준
 - Global Planner의 Global Path 기준
- Local Costmap:
 - 로봇을 중심으로 하는 작은 사각형(rolling window) 형태의 지도
 - 로봇 Frame 기준 (움직일 때마다 지도가 로봇을 따라 이동)
 - Local Planner의 장애물 회피 경로(local path) 및 명령(local control) 용도

계획, Planning

Global Planner

- Global Costmap 기반으로 전체 경로 계획
- 연산량이 많기 때문에 최초 한 번, 이벤트 기반, 또는 느린 주기로 계산 (~1Hz)
- 알고리즘:
 - Dijkstra
 - A*

계획, Planning

Global Planner – Dijkstra

- 출발지부터 모든 격자를 탐색 → 최소 비용의 경로 계산
- 해당 노드까지 모든 grid에 비용 점수 부여
- BFS (Breadth-First Search)
- $f(n) = g(n)$
- 장점: 무조건 최적화된 해답 찾음
- 단점: 연산량 메모리

계획, Planning

Global Planner – A*

- Dijkstra + Heuristic
- 목적지 방향 위주로 우선 탐색
- 해당 노드까지 비용 + 목적지까지의 남은 거리
- 주로 맨해튼 거리 / 유클리드 거리 사용
- $f(n) = g(n) + h(n)$
- 장점: 빠른속도, 연산량, 메모리
- 단점: 비교적 덜 최적화된 경로

행동, Action

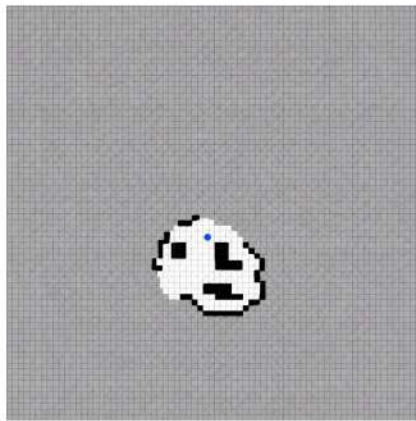
탐험

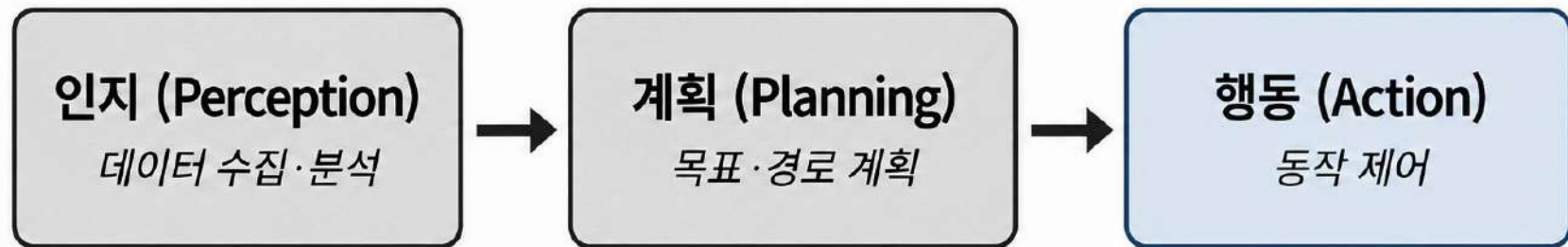


행동, Action

Frontier Exploration 과정

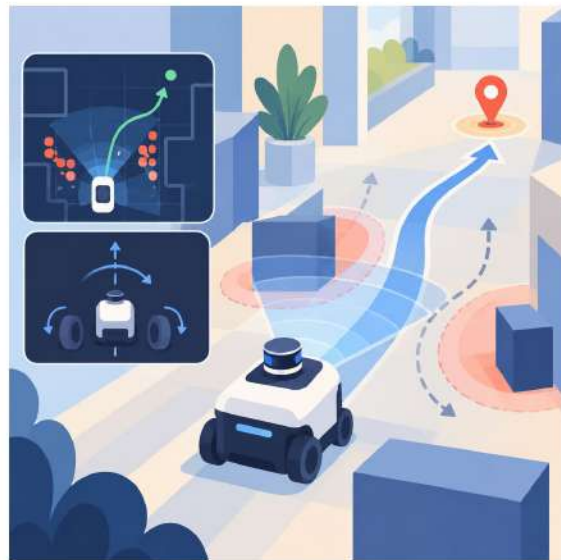
1. Frontier 후보 생성
 - Free cell 주변의 Unknown cell
2. Frontier 후보 중 목표점 선택
 - 거리, 예상 cost, 새롭게 탐색할 수 있는 영역의 크기 등 기준 설립
3. 해당 목표로 Global Path 생성
4. Local Planner로 모터 제어하여 이동





행동, Action

- 계획된 경로에 따른 이동 명령 생성
- 로봇의 속도 및 방향 제어
- 주행 중 위치 및 자세 실시간 보정
- 상황 변화에 따른 로봇 동작 제어



행동, Action

용어 정리

Global Planner :

시작점에서 목표점까지 가기 위해 경로를 계산하기 위한 시스템/모듈

Global Path :

Global Planner가 계산한 시작점에서 목표점까지의 전체적인 이동 경로

Local Planner/Controller :

현재 위치와 주변 환경을 고려하여 가까운 구간에서 움직임을 계산·제어하는 시스템/모듈

Local Path :

Local Planner/Controller가 현재 움직임을 제어하기 위해 사용하는 가까운 구간의 경로

행동, Action

Local Planner (Controller)

- Local Costmap 기반으로 실시간 장애물 회피 경로 계획
- Global Path를 따르기 위해 모터 제어
- 실시간성을 위해 빠른 주기로 계산 (20Hz)
- 알고리즘:
 - DWA
 - TEB
 - MPPI

행동, Action

Decision Making & Reactive Control

Algorithm 1 Simple LiDAR Obstacle Avoidance System

```
1: procedure OBSTACLEAVOIDANCE( $d_F, d_L, d_R, battery$ )
2:   while  $battery > BATTERY\_THRESHOLD$  do
3:     if  $d_F > DISTANCE\_THRESHOLD$  then
4:        $v_L \leftarrow BASE\_SPEED$ 
5:        $v_R \leftarrow BASE\_SPEED$ 
6:     else
7:       if  $d_L > d_R$  then
8:          $v_L \leftarrow -BASE\_SPEED$ 
9:          $v_R \leftarrow BASE\_SPEED$ 
10:      else
11:         $v_L \leftarrow BASE\_SPEED$ 
12:         $v_R \leftarrow -BASE\_SPEED$ 
13:      end if
14:    end if
15:    SETVELOCITY( $v_L, v_R$ )
16:  end while
17: end procedure
```

행동, Action

Decision Making & Reactive Control

Algorithm 2 Yaw-based LiDAR Obstacle Avoidance System

```
1: procedure OBSTACLEAVOIDANCE( $d_F, d_L, d_R, battery$ )
2:   while  $battery > BATTERY\_THRESHOLD$  do
3:      $current\_yaw = GET\_YAW()$ 
4:     if  $STATE = "MOVE"$  then
5:       if  $d_F > DISTANCE\_THRESHOLD$  then
6:          $v_L \leftarrow BASE\_SPEED$ 
7:          $v_R \leftarrow BASE\_SPEED$ 
8:       else
9:          $STATE \leftarrow "TURN"$ 
10:        if  $d_L > d_R$  then
11:           $turn\_dir \leftarrow 1$ 
12:        else
13:           $turn\_dir \leftarrow -1$ 
14:        end if
15:         $target\_yaw \leftarrow current\_yaw + turn\_dir * ANGLE$ 
16:      end if
17:    else if  $STATE = "TURN"$  then
18:      if  $|target\_yaw - current\_yaw| > TOLERANCE$  then
19:         $v_L \leftarrow -turn\_dir * BASE\_SPEED$ 
20:         $v_R \leftarrow turn\_dir * BASE\_SPEED$ 
21:      else
22:         $STATE \leftarrow "MOVE"$ 
23:      end if
24:    end if
25:     $SETVELOCITY(v_L, v_R)$ 
26:  end while
27: end procedure
```

행동, Action

Decision Making & Reactive Control

Algorithm 3 Yaw-based LiDAR Obstacle Avoidance System with Corner Escaping Behavior

```
1: procedure OBSTACLEAVOIDANCE( $d_F, d_L, d_R, battery$ )
2:   while  $battery > BATTERY\_THRESHOLD$  do
3:      $current\_yaw = GETYAW()$ 
4:     if  $turn\_cnt > MAX\_CNT$  then
5:       repeat
6:          $ESCAPEBEHAVIOR()$ 
7:       until  $END\_CONDITION$ 
8:     else if  $STATE = "MOVE"$  then
9:       if  $d_F > DISTANCE\_THRESHOLD$  then
10:         $v_L \leftarrow BASE\_SPEED$ 
11:         $v_R \leftarrow BASE\_SPEED$ 
12:         $turn\_cnt \leftarrow 0$ 
13:      else
14:         $STATE \leftarrow "TURN"$ 
15:         $turn\_cnt \leftarrow turn\_cnt + 1$ 
16:        if  $d_L > d_R$  then
17:           $turn\_dir \leftarrow 1$ 
18:        else
19:           $turn\_dir \leftarrow -1$ 
20:        end if
21:         $target\_yaw \leftarrow current\_yaw + turn\_dir * ANGLE$ 
22:      end if
23:    else if  $STATE = "TURN"$  then
24:      if  $|target\_yaw - current\_yaw| > TOLERANCE$  then
25:         $v_L \leftarrow -turn\_dir * BASE\_SPEED$ 
26:         $v_R \leftarrow turn\_dir * BASE\_SPEED$ 
27:      else
28:         $STATE \leftarrow "MOVE"$ 
29:         $turn\_cnt \leftarrow 0$ 
30:      end if
31:    end if
32:     $SETVELOCITY(v_L, v_R)$ 
33:  end while
34: end procedure
```

행동, Action

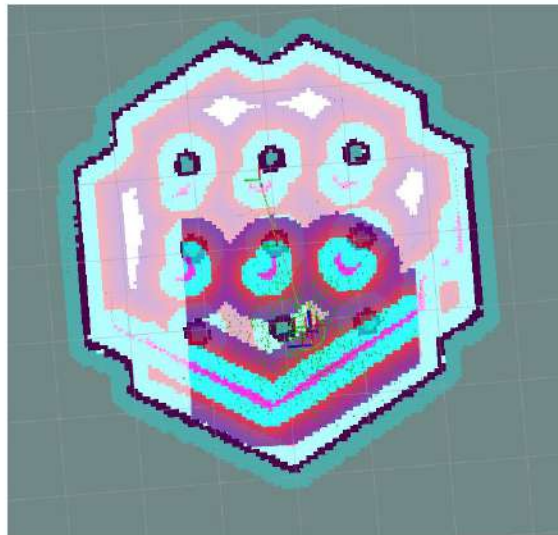
Local Planner (Controller)

- **Local Costmap** 기반으로 실시간 장애물 회피 경로 계획
- Global Path를 따르기 위해 모터 제어
- 실시간성을 위해 빠른 주기로 계산 (20Hz)
- 알고리즘:
 - DWA
 - TEB
 - MPPI

행동, Action

Global Costmap vs. Local Costmap

- Global Costmap:
 - 지도 전체 반영
 - Map Frame 기준
 - Global Planner의 Global Path 기준
- Local Costmap:
 - 로봇을 중심으로 하는 작은 사각형(rolling window) 형태의 지도
 - 로봇 Frame 기준 (움직일 때마다 지도가 로봇을 따라 이동)
 - Local Planner의 장애물 회피 경로(local path) 및 명령(local control) 용도



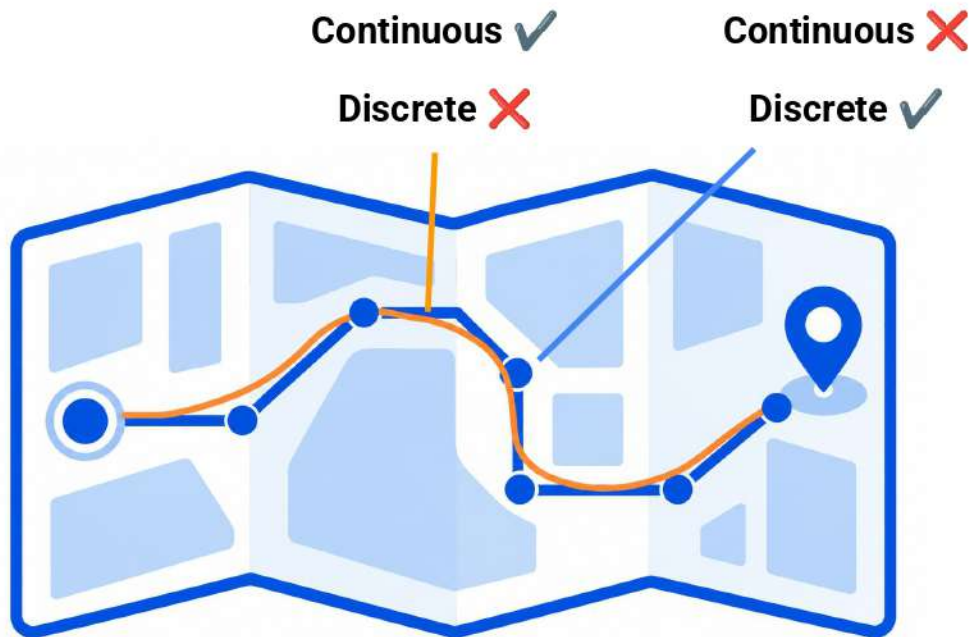
행동, Action

Local Planner (Controller)

- Local Costmap 기반으로 실시간 장애물 회피 경로 계획
- **Global Path**를 따르기 위해 모터 제어
- 실시간성을 위해 빠른 주기로 계산 (20Hz)
- 알고리즘:
 - DWA
 - TEB
 - MPPI

행동, Action

Look-ahead



행동, Action

Look-ahead 과정

1. 로봇의 위치 파악

- 현재 로봇 좌표 기준 가장 가까운 waypoint 계산
- Euclidean Distance 사용
- Nearest Point Search는 라이브러리 사용시 쉽게 해결 가능

```
from scipy.spatial import cKDTree  
from sklearn.neighbors import NearestNeighbors
```

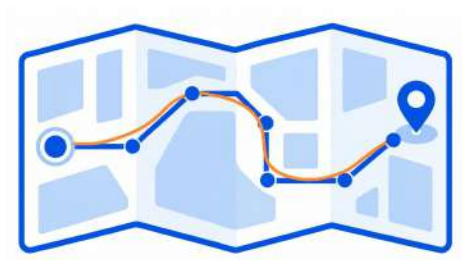


행동, Action

Look-ahead 과정

2. Look-ahead Point 설정

- 거리 기반 목표점 설정
- 직선 거리가 아닌 경로 거리

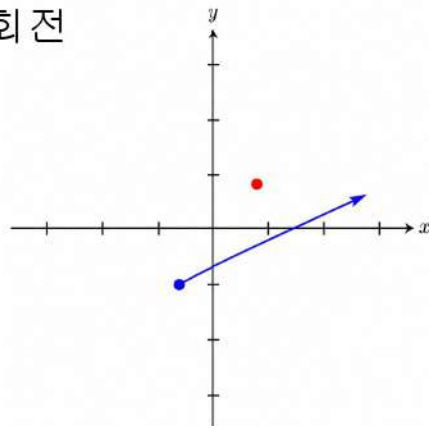


행동, Action

Look-ahead 과정

3. Look-ahead Point 좌표를 로봇 Frame으로 변환

- 로봇 좌표를 원점으로 계산
- 로봇 heading이 0° (+x축 방향)이 되도록 회전
- 계산된 x값은 로봇과의 앞뒤 거리
- 계산된 y값은 로봇과의 좌우 거리

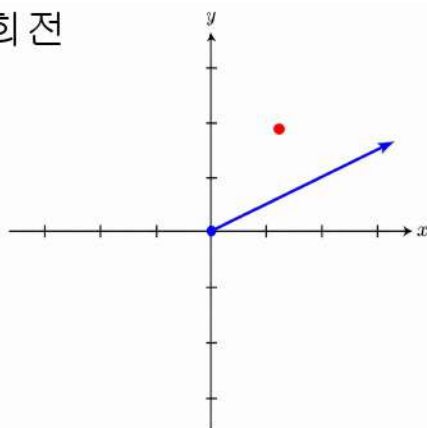


행동, Action

Look-ahead 과정

3. Look-ahead Point 좌표를 로봇 Frame으로 변환

- 로봇 좌표를 원점으로 계산
- 로봇 heading이 0° (+x축 방향)이 되도록 회전
- 계산된 x값은 로봇과의 앞뒤 거리
- 계산된 y값은 로봇과의 좌우 거리

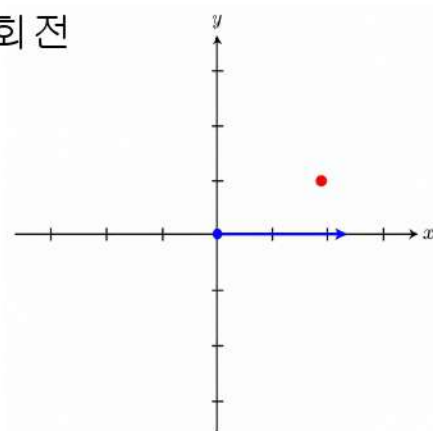


행동, Action

Look-ahead 과정

3. Look-ahead Point 좌표를 로봇 Frame으로 변환

- 로봇 좌표를 원점으로 계산
- 로봇 heading이 0° (+x축 방향)이 되도록 회전
- 계산된 x값은 로봇과의 앞뒤 거리
- 계산된 y값은 로봇과의 좌우 거리



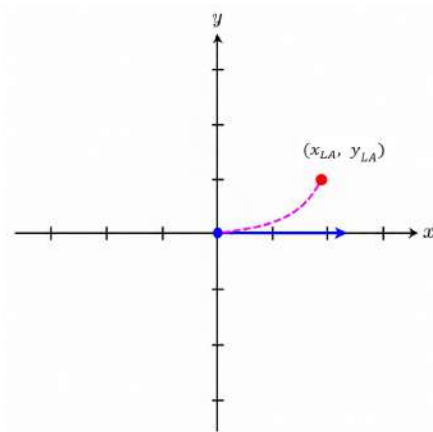
행동, Action

Look-ahead 과정

4. 곡률 계산

- 곡률: 경로가 휘어진 정도 (m^{-1})
- $\kappa > 0$: 좌측, $\kappa < 0$: 우측, $\kappa = 0$: 직선

$$\kappa = \frac{2y_{LA}}{x_{LA}^2 + y_{LA}^2}$$



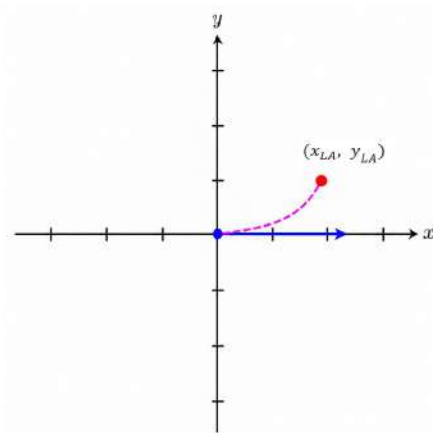
행동, Action

Look-ahead 과정

5. 바퀴 속도 계산 및 전달

- 선속도, 각속도 (v, ω) 계산
- 좌측, 우측 바퀴 속도 (v_r, v_l) 계산

$$\kappa = \frac{2y_{LA}}{x_{LA}^2 + y_{LA}^2} \quad \omega = v\kappa$$



행동, Action

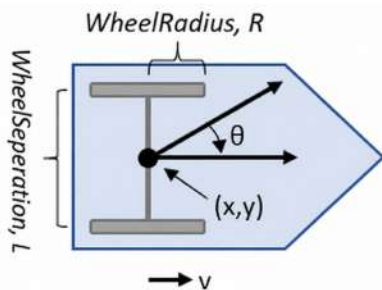
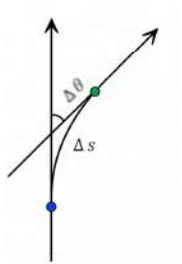
Look-ahead 과정

5. 바퀴 속도 계산 및 전달

- 선속도, 각속도 (v, ω) 계산
- 좌측, 우측 바퀴 속도 (v_r, v_l) 계산

$$\Delta s = \frac{d_r + d_l}{2}$$

$$\Delta \theta = \frac{d_r - d_l}{L}$$



행동, Action

Look-ahead 과정

5. 바퀴 속도 계산 및 전달

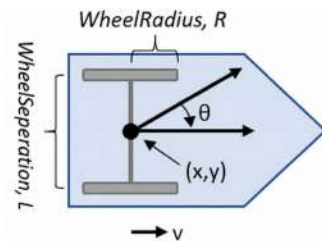
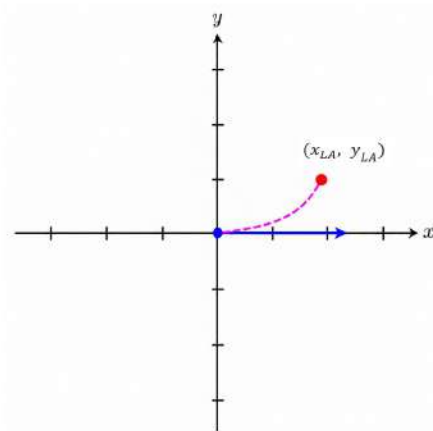
- 선속도, 각속도 (v, ω) 계산
- 좌측, 우측 바퀴 속도 (v_r, v_l) 계산

$$v = \frac{v_r + v_l}{2}$$

$$v_r = v + \frac{\omega L}{2}$$

$$\omega = \frac{v_r - v_l}{L}$$

$$v_l = v - \frac{\omega L}{2}$$



행동, Action

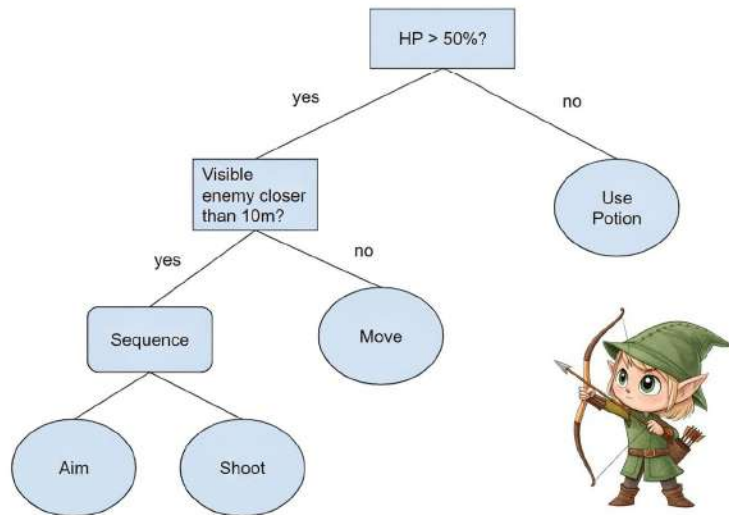
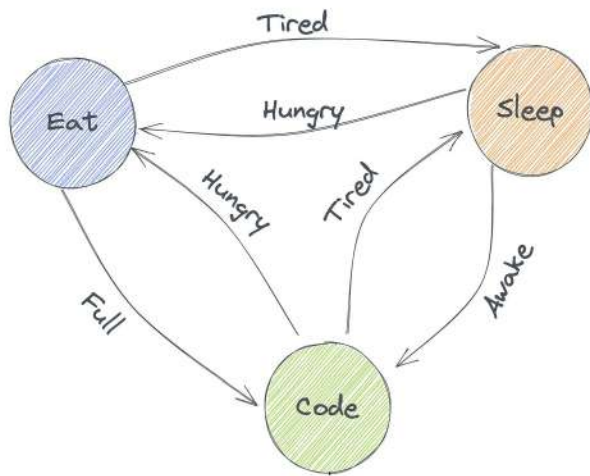
Look-ahead Distance 조정

- 너무 긴 Look-ahead Distance
 - 경로에서 크게 벗어날 가능성
 - 급커브 구간에서 경로 안쪽을 가로지르는 현상
- 너무 짧은 Look-ahead Distance
 - 커브에서 불안정한 주행
 - 지그재그 움직임 발생



행동, Action

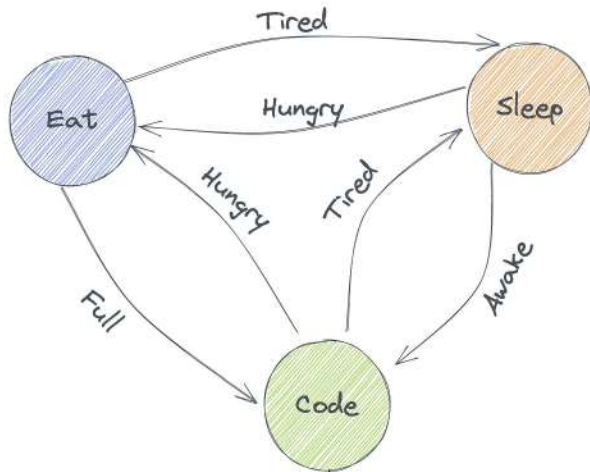
Decision Making



행동, Action

Decision Making: Finite State Machine (FSM)

- 그래프 구조
- State(상태) & Transition(전이)
- 동작 프로세스가 단순하고 순차적인 경우
- 상태의 개수가 적고, 서로 복잡하게 얽힐 일이 없는 경우
- 상태와 조건/전이가 늘어날수록 복잡도가 기하급수적으로 증가

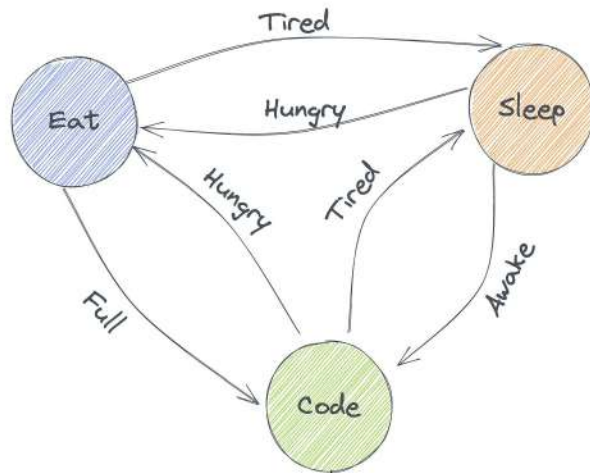


행동, Action

Decision Making: Finite State Machine (FSM)

Algorithm 2 Yaw-based LiDAR Obstacle Avoidance System

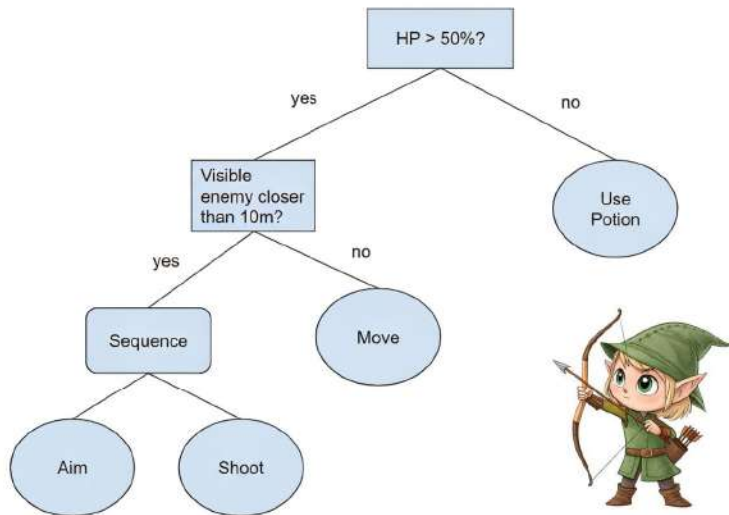
```
1: procedure OBSTACLEAVOIDANCE( $d_F, d_L, d_R, battery$ )
2:   while  $battery > BATTERY\_THRESHOLD$  do
3:      $current\_yaw = GET\_YAW()$ 
4:     if  $STATE = "MOVE"$  then
5:       if  $d_F > DISTANCE\_THRESHOLD$  then
6:          $v_L \leftarrow BASE\_SPEED$ 
7:          $v_R \leftarrow BASE\_SPEED$ 
8:       else
9:          $STATE \leftarrow "TURN"$ 
10:        if  $d_L > d_R$  then
11:           $turn\_dir \leftarrow 1$ 
12:        else
13:           $turn\_dir \leftarrow -1$ 
14:        end if
15:         $target\_yaw \leftarrow current\_yaw + turn\_dir * ANGLE$ 
16:      end if
17:    else if  $STATE = "TURN"$  then
18:      if  $|target\_yaw - current\_yaw| > TOLERANCE$  then
19:         $v_L \leftarrow -turn\_dir * BASE\_SPEED$ 
20:         $v_R \leftarrow turn\_dir * BASE\_SPEED$ 
21:      else
22:         $STATE \leftarrow "MOVE"$ 
23:      end if
24:    end if
25:     $SET\_VELOCITY(v_L, v_R)$ 
26:  end while
27: end procedure
```



행동, Action

Decision Making: Behavior Tree

- 트리 구조
- Node로 구성
- 다양한 조건과 예외 처리가 복잡하게 얽힌 경우
- 조건에 따라 동작을 유연하게 선택하고 실행
- 노드의 추가·변경이 기존 로직에 미치는 영향이 적음
- 복잡한 행동과 예외 상황을 구조적으로 관리 가능

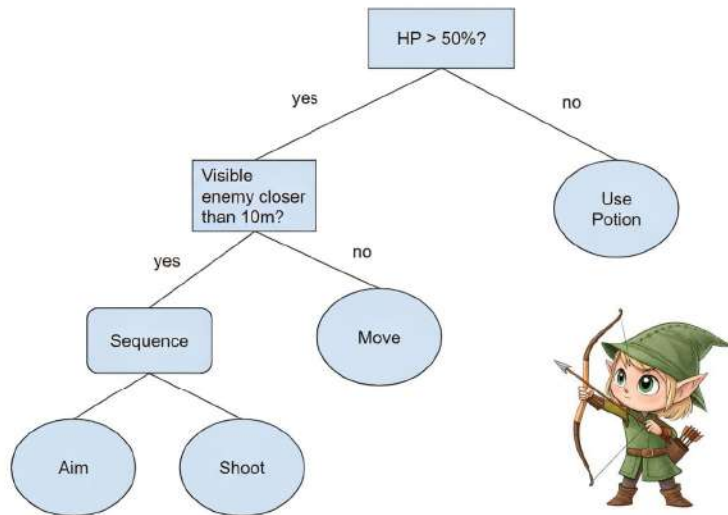


행동, Action

Decision Making: Behavior Tree

트리 용어

- Node
- Root
- Branch
- Leaf
- Parent/Child
- Grandparent/Grandchild
- Uncle/Nephew
- Sibling
- Blackboard

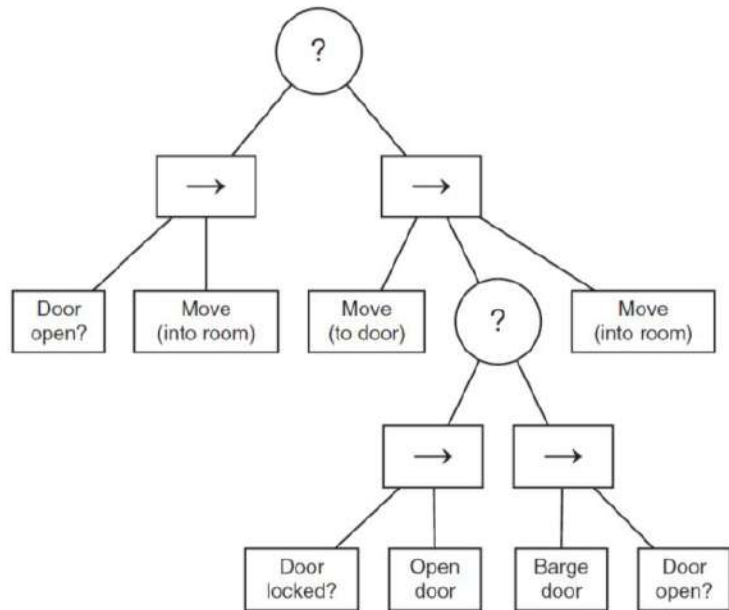


행동, Action

Decision Making: Behavior Tree

노드 종류 - 실행 노드 (Leaf)

- Task/Action
- Condition

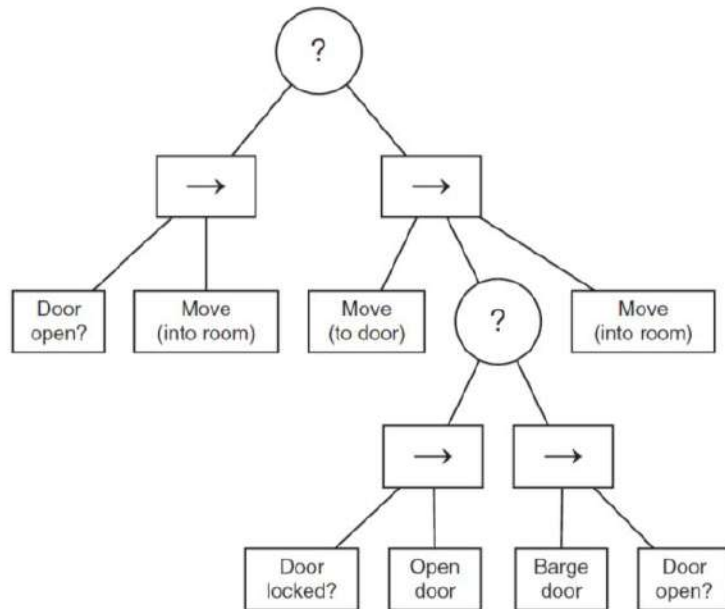


행동, Action

Decision Making: Behavior Tree

노드 종류 - 제어 노드 (Internal/Inner Node)

- Composite:
 - Selector
 - Sequence
 - Parallel
- Decorator:
 - Inverter
 - Loop
 - Repeater
 - Succeder
 - Failer

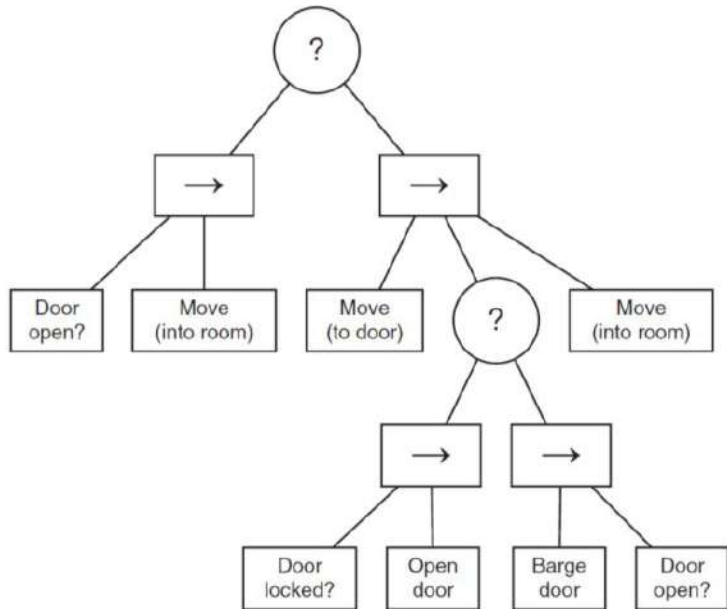


행동, Action

Decision Making: Behavior Tree

Behavior Tree 노드의 특징

- Success / Failure / Running 상태 반환
- Child의 결과에 따라 다음 실행 흐름 결정
- S/F/R 결과는 parent로 Bubble Up
- Tick 단위로 평가 및 실행

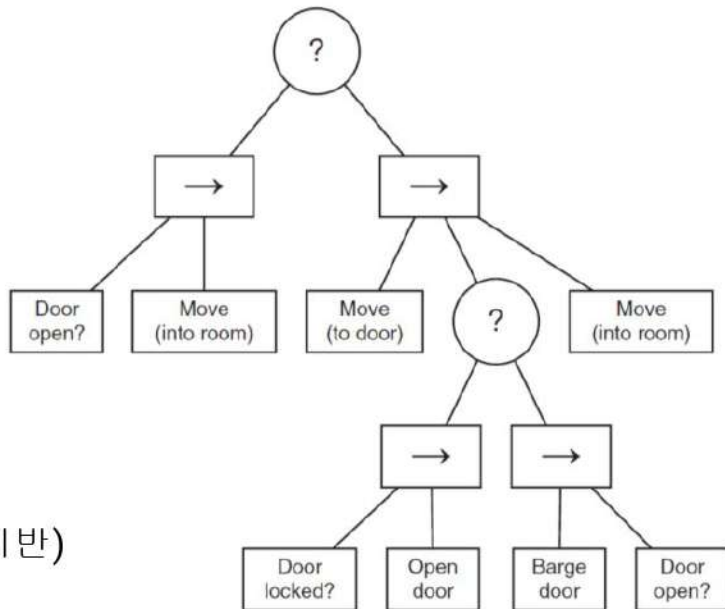


행동, Action

Decision Making: Behavior Tree

1. Selector

- 왼쪽 → 오른쪽 순서로 Child 실행
- [S]: 하나라도 성공한 경우
- [F]: 전부 실패한 경우
- [R]: 하나라도 Running인 경우
- 다음 Tick에서 Running Child부터 검사 (메모리 기반)

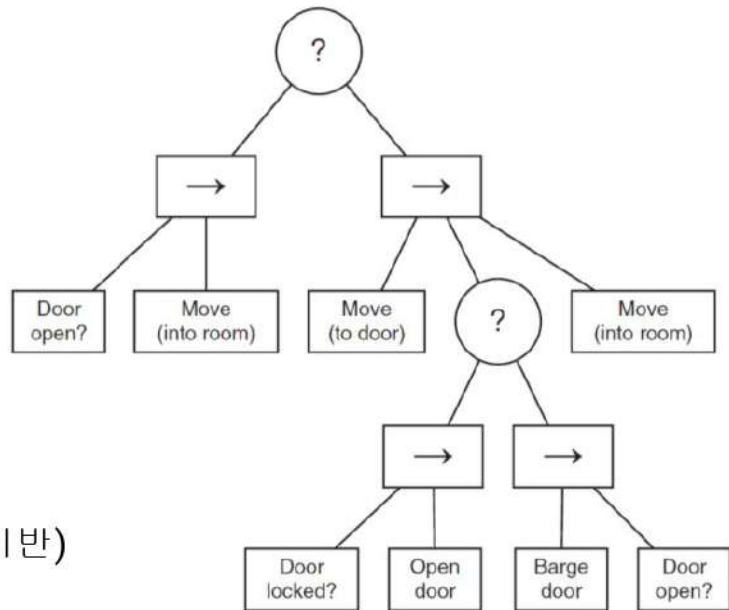


행동, Action

Decision Making: Behavior Tree

2. Sequence

- 왼쪽 → 오른쪽 순서로 Child 실행
- [S]: 전부 성공한 경우
- [F]: 하나라도 실패한 경우
- [R]: 하나라도 Running인 경우
- 다음 Tick에서 Running Child부터 검사 (메모리 기반)

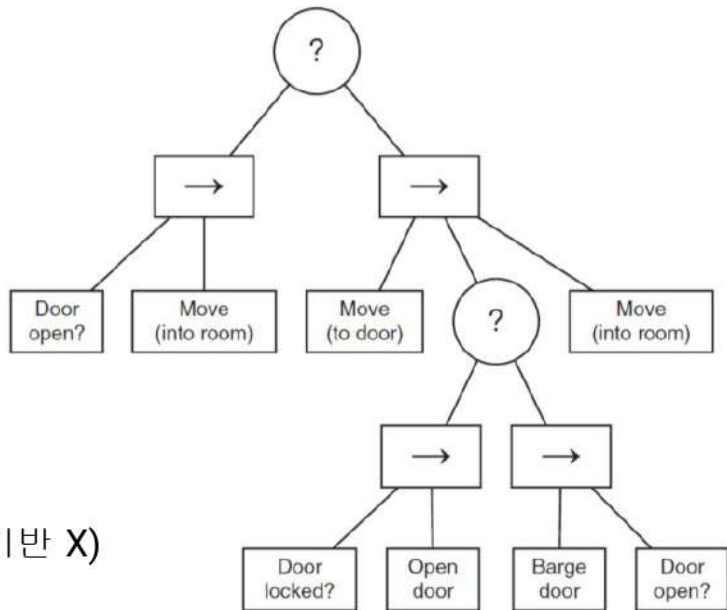


행동, Action

Decision Making: Behavior Tree

3. Parallel

- 모든 Child 실행 (순서 의미 없음)
- [S]: 성공한 Child 수 $\geq$ Success Count
- [F]: 실패한 Child 수 $\geq$ Failure Count
- [R]: S/F 조건 충족하지 않은 경우
- 매 Tick마다 모든 Child 다시 실행/평가 (메모리 기반 X)

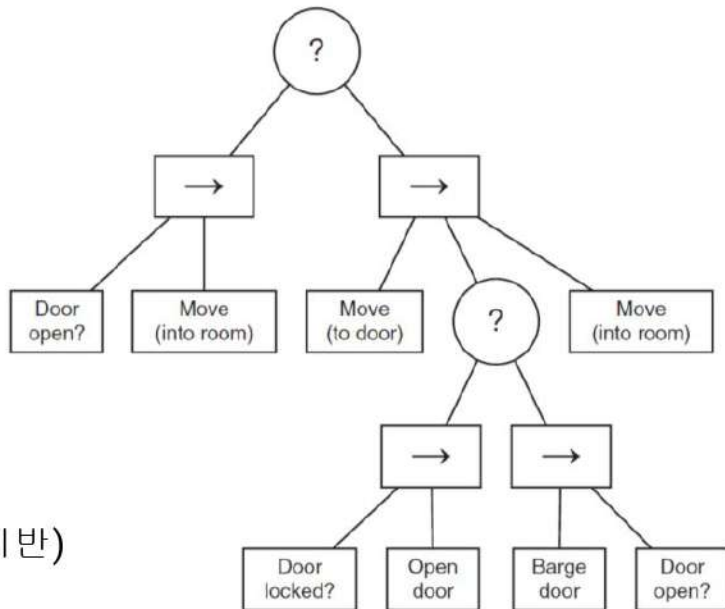


행동, Action

Decision Making: Behavior Tree

4. Fallback (Selector)

- 왼쪽 → 오른쪽 순서로 Child 실행
- [S]: 하나라도 성공한 경우
- [F]: 전부 실패한 경우
- [R]: 하나라도 Running인 경우
- 다음 Tick에서 Running Child부터 검사 (메모리 기반)

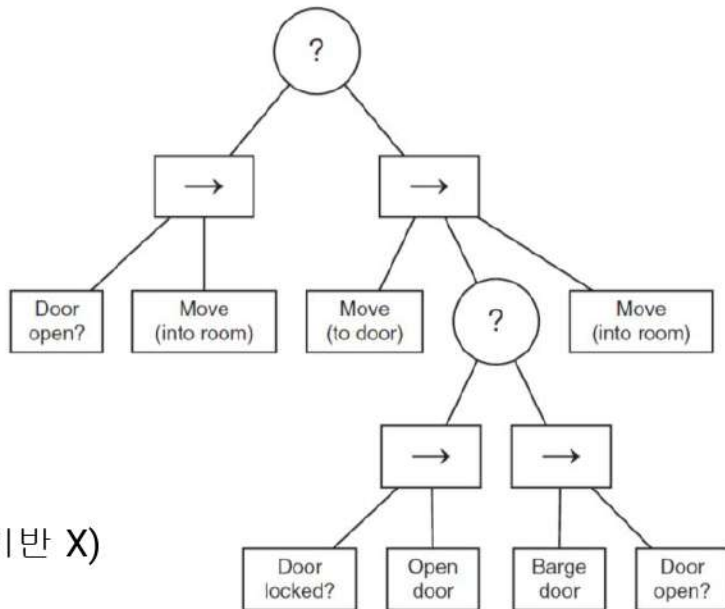


행동, Action

Decision Making: Behavior Tree

5. Reactive Fallback (Selector)

- 왼쪽 → 오른쪽 순서로 Child 실행
- [S]: 하나라도 성공한 경우
- [F]: 전부 실패한 경우
- [R]: 하나라도 Running인 경우
- 매 Tick마다 모든 Child 다시 실행/평가 (메모리 기반 X)

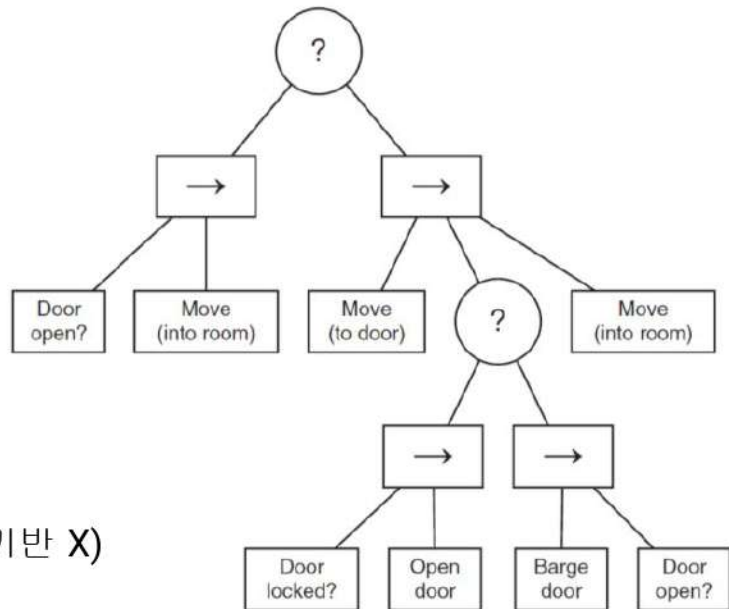


행동, Action

Decision Making: Behavior Tree

6. Reactive Sequence

- 왼쪽 → 오른쪽 순서로 Child 실행
- [S]: 전부 성공한 경우
- [F]: 하나라도 실패한 경우
- [R]: 하나라도 Running인 경우
- 매 Tick마다 모든 Child 다시 실행/평가 (메모리 기반 X)

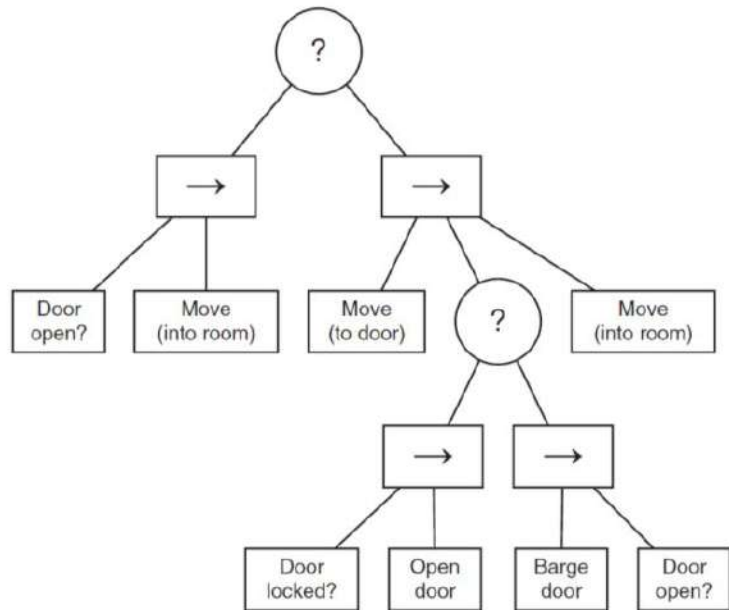


행동, Action

Decision Making: Behavior Tree

7. Pipeline Sequence

- Reactive Sequence와 동일 로직
- 차이: Running Child 만나도 기존 수행 작업 진행
- Latency 존재 시 중단 없이 실행 Pipeline을 진행
- Camera Input [S] → Detection [R] → Action [-]
- Camera Input [R] → Detection [S] → Action [S]

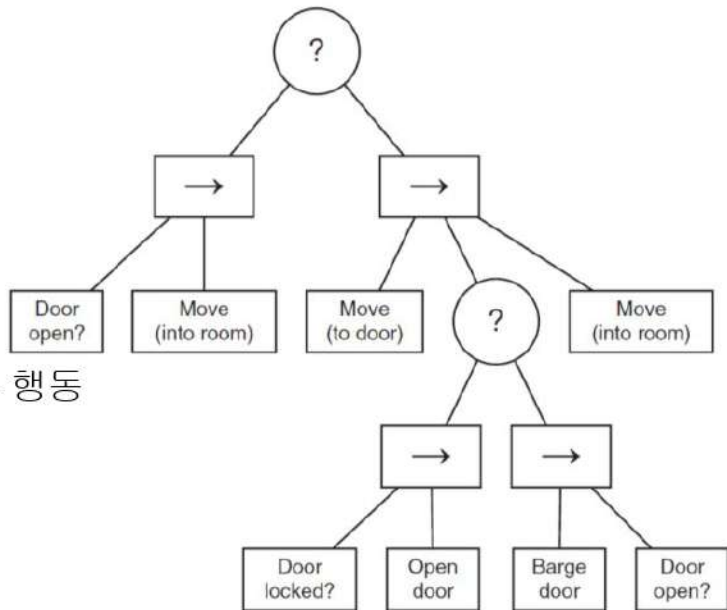


행동, Action

Decision Making: Behavior Tree

8. Recovery Node

- Children 2개로 제한
- Child #1 (Main Action): 주 행동
- Child #2 (Recovery Action): 실패 시 해결할 회복 행동
- 예) 주행 - 제자리 회전
- [S]: Child #1이 Success 반환
- [F]: Child #2 (회복) 실패
- [F]: 재시도 횟수 초과

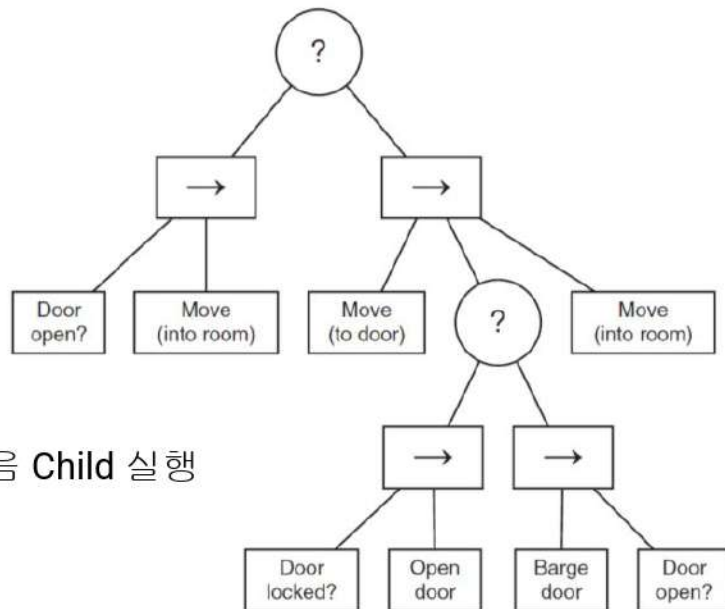


행동, Action

Decision Making: Behavior Tree

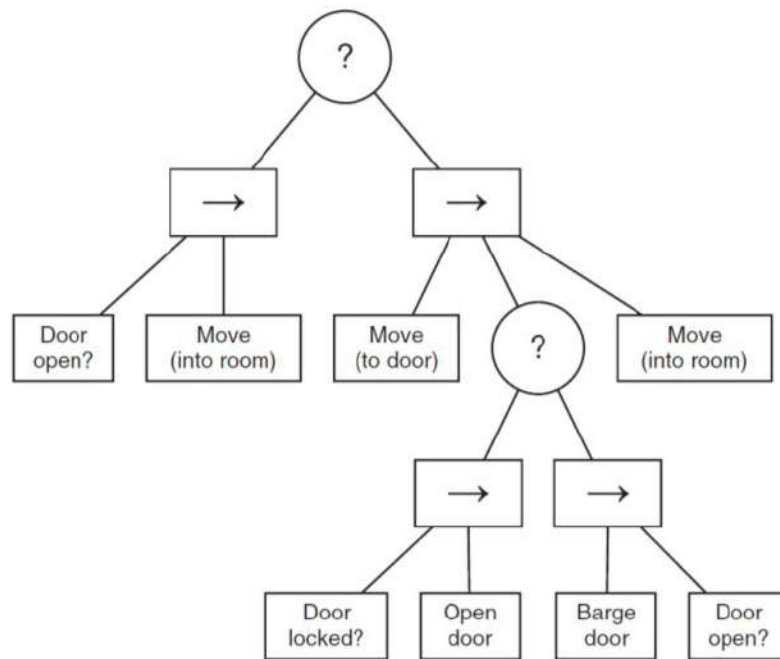
9. Round Robin

- 일반 Selector/Sequence와 유사
- 공통점: 다음 Tick에서 Running Child부터 검사
- 공통점: Child가 [R] 반환 시 부모도 [R]
- 차이점: Child가 [S]/[F] 반환 시 다음 Tick에서 다음 Child 실행



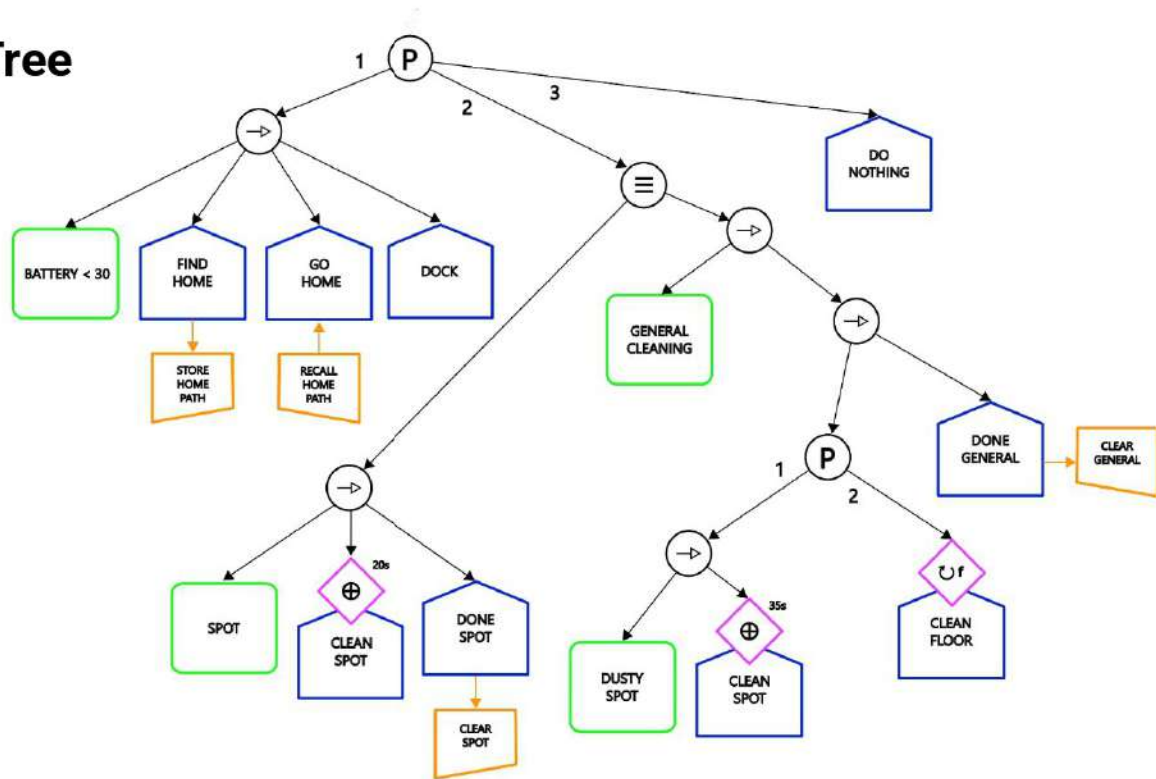
행동, Action

Decision Making: Behavior Tree



행동, Action

Decision Making: Behavior Tree



행동, Action

Decision Making: Finite State Machine (FSM)

```
while True:
    if state == IDLE:
        if IsBatteryLow():
            state = GO_HOME
        elif IsDirty():
            state = CLEANING

    elif state == CLEANING:
        Clean()
        state = IDLE

    elif state == GO_HOME:
        GoHome()
        state = DOCK

    elif state == DOCK:
        Dock()
        state = IDLE
```

행동, Action

Decision Making: Behavior Tree

```
tree = Parallel([  
    Sequence([  
        IsBatteryLow(),  
        GoHome(),  
        Dock()  
    ]),  
    Selector([  
        Sequence([  
            IsDirty(),  
            Clean()  
        ]),  
        Idle()  
    ])  
])
```

